

BiteFixes Technical Implementation Blueprint

Volume 01

Platform Implementation Foundation (PIF)

Version: 1.0

Classification: Technical Architecture Documentation

Status: Draft

Table of Contents

1.1 Purpose

1.2 Implementation Philosophy

1.3 Technical Architecture Principles

1.4 Platform Implementation Overview

1.5 Technology Stack Definition

1.6 Repository Architecture

1.7 Backend Architecture

1.8 Database Architecture

1.9 AI Engine Implementation Strategy

1.10 API Architecture

1.11 Frontend and Channel Architecture

1.12 Infrastructure Implementation

1.13 Security Implementation Foundation

1.14 Development Workflow

1.15 Implementation Roadmap

1.16 Volume Summary

1.1 Purpose

Overview

The BiteFixes Technical Implementation Blueprint defines the practical implementation model required to transform the Enterprise Architecture into a functional SaaS platform.

While the Enterprise Architecture Manual defined the strategic and conceptual foundation, this blueprint defines:

- Real software components.
- Repository organization.
- Backend structure.
- Database implementation.
- AI engine development.
- API design.
- Deployment model.
- Engineering workflow.

Technical Objective

The objective is to build BiteFixes as:

A scalable, secure, multilingual SaaS platform powered by Bitey AI as the central intelligence layer.

Implementation Scope

This blueprint covers:

Backend Services

Database Layer

AI Engine

API Layer

Web Platform

Mobile Applications

Integrations

Infrastructure

Security

Deployment

Operations

1.2 Implementation Philosophy

Overview

BiteFixes development follows enterprise engineering principles.

The platform must be:

- Modular.
- Scalable.
- Maintainable.
- Secure.
- Testable.
- Cloud-ready.
- AI-integrated.

Core Development Principles

Principle 1 — Backend First

The backend represents the central nervous system of BiteFixes.

All channels communicate through the backend.

Architecture:

WhatsApp

Website

Mobile App

External Systems

|



BiteFixes Backend

|



Bitey AI Engine

|



Database

Principle 2 — Bitey as Central Intelligence

Bitey is not a separate chatbot.

Bitey is the intelligence layer responsible for:

- Understanding requests.
- Detecting intent.
- Accessing knowledge.
- Maintaining memory.
- Creating automation.
- Coordinating workflows.

Principle 3 — Database as Platform Memory

Supabase acts as the central data platform.

Stores:

- Customers.
- Companies.
- Tickets.
- Conversations.
- Knowledge.
- AI memory.
- Services.
- Operational data.

Principle 4 — Channel Independence

Channels are interchangeable.

Example:

A customer conversation can start through:

WhatsApp

↓

Website Chat

↓

Mobile Application

without changing the intelligence layer.

1.3 Technical Architecture Principles

Modular Architecture

The system is divided into independent modules.

Example:

Authentication

Customers

Tickets

Knowledge

AI Engine

Notifications

Integrations

Analytics

Service-Oriented Design

Each business capability is isolated.

Example:

Customer Service

Ticket Service

Knowledge Service

AI Service

API-First Development

Every important capability is exposed through APIs.

Benefits:

- Mobile compatibility.
- External integrations.
- Future expansion.

Security by Design

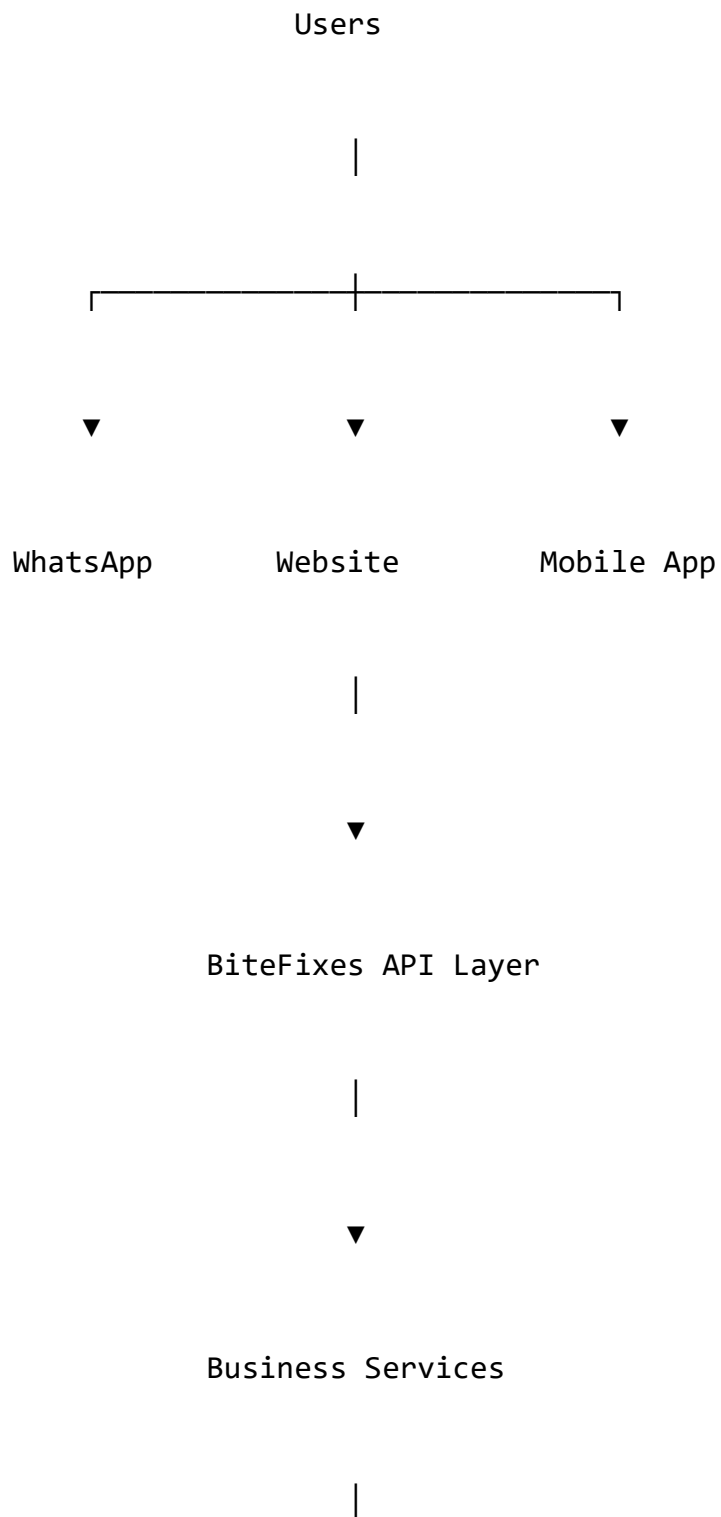
Security is implemented from the beginning.

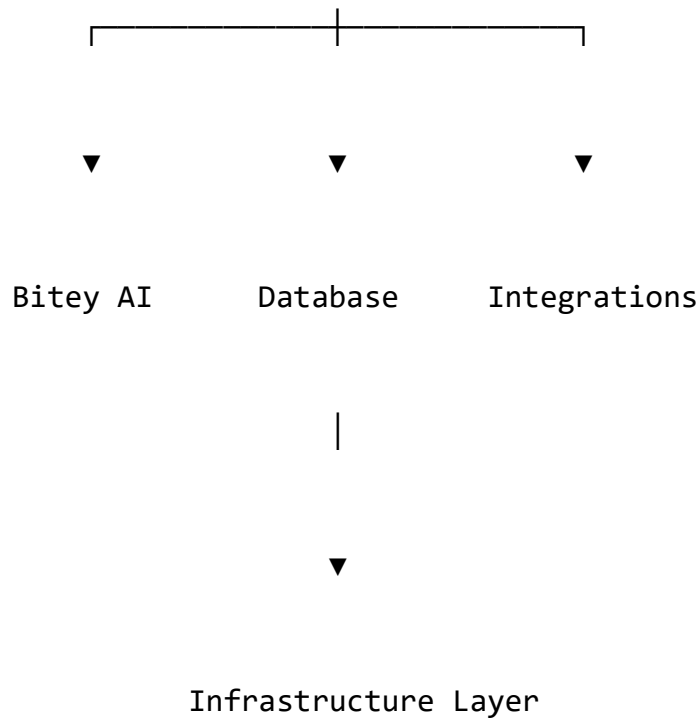
Includes:

- Authentication.
- Authorization.
- Data isolation.
- Encryption.
- Auditing.

1.4 Platform Implementation Overview

The complete BiteFixes platform architecture:





1.5 Technology Stack Definition

Backend

Primary technology:

Python

FastAPI

Pydantic

Uvicorn

SQL/ORM Layer

Responsibilities:

- Business logic.
- APIs.

- Authentication.
- AI coordination.
- Integrations.

Database

Primary platform:

Supabase

PostgreSQL

Storage

Authentication

Realtime Services

Responsibilities:

- Persistent data.
- User management.
- AI memory.
- Application state.

Artificial Intelligence

Bitey Engine:

Components:

Intent Detection

Knowledge Retrieval

Conversation Memory

Response Generation

Automation Engine

Frontend Channels

Supported:

WordPress Website

Web Chat

Flutter Mobile Application

WhatsApp Integration

Infrastructure

Target production:

Cloud Hosting

Containerized Services

CI/CD Pipeline

Monitoring

Backup System

1.6 Repository Architecture

Recommended repository structure:

bitefixes-platform/

|

├─ backend/

|

├─ app/

|

|

├─ core/

├─ services/

├─ routers/

├─ models/

├─ schemas/

├─ database/

└─ integrations/

|

├─ frontend/

|

├─ mobile/

|

├─ infrastructure/

```
|
|
├─ documentation/
|
|
├─ tests/
|
|
└─ scripts/
```

Backend Structure

Detailed:

```
backend/app/
|
├─ main.py
|
├─ config.py
|
├─ database/
|   └─ supabase.py
|   └─ connection.py
|
├─ core/
|   └─ bitey.py
|   └─ security.py
|   └─ settings.py
```

```
├─ services/
|   ├─ cliente_service.py
|   ├─ ticket_service.py
|   ├─ conocimiento_service.py
|   ├─ chat_memory.py
|   └─ intent_service.py
```

```
├─ routers/
|   ├─ chat.py
|   ├─ clientes.py
|   ├─ tickets.py
|   └─ knowledge.py
```

```
├─ models/
```

```
├─ schemas/
```

```
└─ utils/
```

1.7 Backend Architecture

Overview

The backend is the central execution layer.

Responsibilities:

- Receive requests.
- Validate information.
- Execute business rules.
- Communicate with Bitey.
- Store information.

Backend Flow

Incoming Message

|



API Router

|



Business Service

|



Bitey Engine

|



Database

|



Response

Backend Modules

Authentication Module

Handles:

- Users.
- Sessions.
- Permissions.

Customer Module

Handles:

- Customers.
- Companies.
- Profiles.

Ticket Module

Handles:

- Incidents.
- Repairs.
- Workflow.

Knowledge Module

Handles:

- Technical information.
- Solutions.
- AI knowledge.

AI Module

Handles:

- Intent detection.
- Memory.
- Response generation.

1.8 Database Architecture

Overview

The database represents the operational memory of BiteFixes.

Core Tables

clientes

empresas

usuarios

tickets

servicios

conversaciones

historial_chats

base_conhecimento

sinonimos_ia

ia_logs

Data Relationships

Example:

Company

|

├─ Users

├─ Customers

├─ Tickets

└─ Conversations

1.9 AI Engine Implementation Strategy

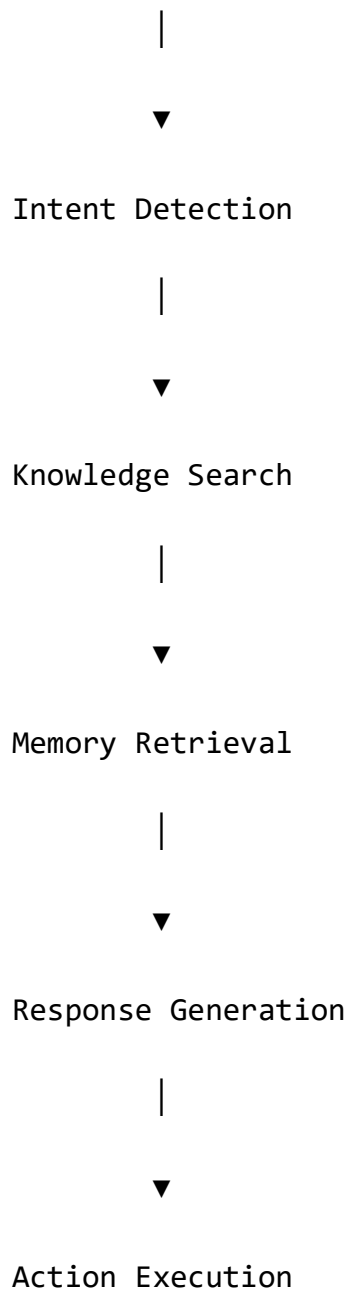
Bitey implementation:

Message Received

|

▼

Text Normalization



1.10 API Architecture

Main API groups:

/auth

/customers

/tickets

/services

/chat

/knowledge

/analytics

Example:

POST /chat

Input:

message

customer_id

channel

Output:

response

intent

confidence

ticket_status

1.11 Frontend and Channel Architecture

All channels connect to the same backend.

Mobile App



Website



WhatsApp



BiteFixes API



Bitey AI

1.12 Infrastructure Implementation

Initial production architecture:

Users



Cloud Platform



FastAPI Backend



Supabase



External Services

1.13 Security Implementation Foundation

Security components:

Authentication

Authorization

Encryption

Audit Logs

Access Control

Backup Protection

1.14 Development Workflow

Development lifecycle:

Planning



Development



Testing

↓

Review

↓

Deployment

↓

Monitoring

↓

Improvement

1.15 Implementation Roadmap

Phase 1

Foundation:

- Backend.
- Database.
- Bitey Core.
- API.

Phase 2

Expansion:

- WhatsApp.
- Website integration.
- Mobile application.

Phase 3

Enterprise:

- Multi-tenant.
- Analytics.
- Advanced AI.

1.16 Volume Summary

The **Platform Implementation Foundation** establishes the technical foundation required to convert BiteFixes architecture into a real production SaaS platform.

This volume defines:

- Development philosophy.
- Technology stack.
- Repository structure.
- Backend architecture.
- Database foundation.
- AI implementation model.
- API strategy.
- Infrastructure approach.

This document becomes the technical starting point for all future BiteFixes engineering work.

End of Volume 01

**BiteFixes Technical Implementation
Blueprint**

**BiteFixes Technical Implementation
Blueprint**

Volume 02

Backend Engineering Architecture (BEA)

Version: 1.0

Classification: Technical Implementation Documentation

Status: Draft

Table of Contents

2.1 Purpose

2.2 Backend Architecture Vision

2.3 FastAPI Application Architecture

2.4 Backend Layer Model

2.5 Project Directory Structure

2.6 Core Application Components

2.7 Router Architecture

2.8 Service Layer Architecture

2.9 Data Access Architecture

2.10 Business Logic Separation

2.11 Dependency Injection Model

2.12 Configuration Management

2.13 Error Handling Strategy

2.14 Logging Architecture

2.15 Testing Architecture

2.16 Backend Development Standards

2.17 Volume Summary

2.1 Purpose

Overview

The Backend Engineering Architecture defines the internal structure and engineering standards of the BiteFixes backend platform.

The objective is to create a backend capable of supporting:

- SaaS multi-tenancy.
- AI-powered customer support.
- Multiple communication channels.
- Enterprise integrations.
- High scalability.

Backend Objective

The backend must operate as:

The central execution engine connecting customers, channels, business processes, and Bitey AI intelligence.

2.2 Backend Architecture Vision

Overview

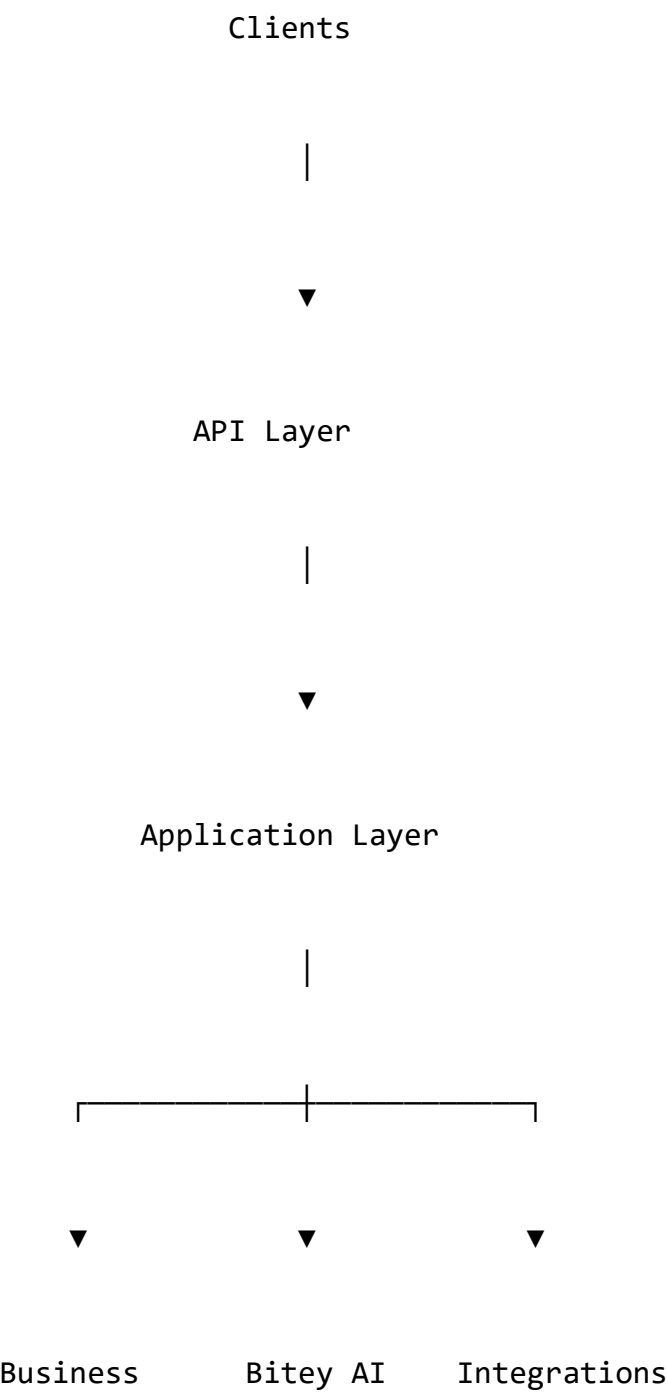
The BiteFixes backend follows a layered architecture.

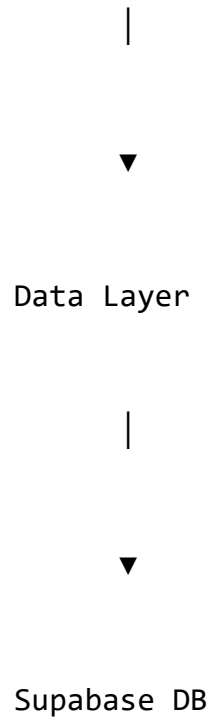
The system separates:

- API communication.

- Business logic.
- Data access.
- AI processing.
- External integrations.

Backend High-Level Model





2.3 FastAPI Application Architecture

Overview

FastAPI is the primary backend framework.

Reasons:

- High performance.
- Async support.
- Automatic OpenAPI documentation.
- Strong typing.
- Modern Python ecosystem.

FastAPI Application Flow

HTTP Request



FastAPI Router



Request Validation



Service Execution



Database / AI Processing



Response Model

Main Application Entry Point

File:

`app/main.py`

Responsibilities:

- Create FastAPI instance.
- Register routers.
- Configure middleware.
- Initialize services.
- Configure startup events.

Example structure:

```
app = FastAPI(  
    title="BiteFixes Backend",  
    version="1.0.0"  
)
```

2.4 Backend Layer Model

BiteFixes uses a six-layer architecture.

Layer 1 — API Layer

Purpose:

Communication with external clients.

Contains:

- Routers.
- Request validation.
- Response formatting.

Location:

app/routers/

Layer 2 — Application Layer

Purpose:

Coordinates business workflows.

Contains:

- Service orchestration.
- Process execution.

Location:

app/services/

Layer 3 — Intelligence Layer

Purpose:

AI processing.

Contains:

- Bity engine.
- Intent detection.
- Knowledge retrieval.
- Memory.

Location:

app/core/

Layer 4 — Domain Layer

Purpose:

Business entities.

Contains:

- Models.
- Schemas.
- Data structures.

Locations:

app/models/

app/schemas/

Layer 5 — Data Layer

Purpose:

Database communication.

Contains:

- Supabase client.
- Queries.
- Repository functions.

Location:

app/database/

Layer 6 — Integration Layer

Purpose:

External communication.

Contains:

- WhatsApp API.
- WordPress API.
- Payment providers.
- Email services.

Location:

app/integrations/

2.5 Project Directory Structure

Production structure:

backend/

├─ app/

|

├─ main.py

├─ config.py

|

├─ core/

- | └─ bitey.py
- | └─ security.py
- | └─ exceptions.py
- | └─ constants.py

- |

- | └─ routers/

- | | └─ auth.py
- | | └─ chat.py
- | | └─ clientes.py
- | | └─ tickets.py
- | | └─ servicios.py
- | | └─ knowledge.py

- |

- | └─ services/

- | | └─ cliente_service.py
- | | └─ ticket_service.py
- | | └─ conocimiento_service.py
- | | └─ historial_service.py
- | | └─ chat_memory.py
- | | └─ intent_service.py

| └─ notification_service.py

|

|─ database/

| └─ supabase.py

| └─ repositories/

|

|─ models/

|

|─ schemas/

|

|─ integrations/

| └─ whatsapp.py

| └─ wordpress.py

| └─ email.py

|

|─ tests/

|

|─ utils/

2.6 Core Application Components

Bitey Engine

File:

`app/core/bitey.py`

Central AI coordinator.

Responsibilities:

- Receive messages.
- Analyze context.
- Detect intention.
- Search knowledge.
- Generate response.
- Trigger actions.

Flow:

User Message



Bitey Engine



Intent Service



Knowledge Service



Memory Service



Response

Security Core

File:

`app/core/security.py`

Responsibilities:

- Authentication.
- Token handling.
- Permissions.

Configuration Core

File:

`app/config.py`

Responsibilities:

- Environment variables.
- Application settings.
- External service configuration.

2.7 Router Architecture

Routers expose backend capabilities.

Chat Router

File:

routers/chat.py

Responsibilities:

- Receive conversations.
- Connect channels.
- Execute Bitey.

Example:

POST /chat

Customer Router

Responsibilities:

- Customer creation.
- Customer retrieval.
- Profile management.

Ticket Router

Responsibilities:

- Ticket creation.
- Status updates.
- Assignment.

Knowledge Router

Responsibilities:

- Knowledge management.
- AI training data.

2.8 Service Layer Architecture

Services contain business logic.

They should NOT:

- Handle HTTP.
- Know frontend details.
- Contain UI logic.

Example:

Router:

POST /tickets

calls:

```
ticket_service.create_ticket()
```

Service Responsibilities

Cliente Service

Handles:

- Customer lifecycle.
- Customer lookup.
- Customer data.

Ticket Service

Handles:

- Ticket workflow.
- Priority.
- Status.

Knowledge Service

Handles:

- Search.
- Retrieval.
- AI information.

Intent Service

Handles:

- Message classification.
- Intent scoring.
- Synonym matching.

2.9 Data Access Architecture

Overview

Database access is isolated from business logic.

Repository Pattern

Example:

Service

|



Repository

|



Supabase

Benefits:

- Easier testing.
- Database independence.
- Cleaner architecture.

2.10 Business Logic Separation

Rules:

Routers:

DO:

- Receive requests.
- Return responses.

DO NOT:

- Execute complex logic.
- Query database directly.

Services:

DO:

- Execute workflows.
- Apply rules.
- Coordinate components.

Database:

DO:

- Store.
- Retrieve.

2.11 Dependency Injection Model

FastAPI dependencies manage:

- Database clients.
- Authentication.

- Services.

Example:

Request

↓

Dependency

↓

Service

↓

Response

Benefits:

- Cleaner code.
- Easier testing.
- Better scalability.

2.12 Configuration Management

Configuration uses environment variables.

Example:

SUPABASE_URL

SUPABASE_KEY

DATABASE_URL

SECRET_KEY

OPENAI_API_KEY

WHATSAPP_TOKEN

Rules:

Never store secrets inside source code.

2.13 Error Handling Strategy

Standard error model:

```
{
  "success": false,
  "error": "resource_not_found",
  "message": "Customer does not exist"
}
```

Error categories:

- Validation errors.
- Authentication errors.
- Database errors.
- Integration errors.
- AI errors.

2.14 Logging Architecture

Logs capture:

- Requests.
- Errors.

- AI decisions.
- Performance.

Log categories:

Application Logs

Security Logs

AI Logs

Integration Logs

Audit Logs

2.15 Testing Architecture

Testing levels:

Unit Tests

Validate:

- Functions.
- Services.
- Business rules.

Integration Tests

Validate:

- Database.
- APIs.
- External services.

End-to-End Tests

Validate:

Complete user flows.

2.16 Backend Development Standards

Standards:

Code Language

Backend code:

English.

Naming Convention

Python:

`snake_case`

Examples:

`create_ticket()`

`customer_service`

Documentation

Required:

- Docstrings.
- Architecture notes.
- API documentation.

Quality Requirements

Every module should be:

- Testable.
- Documented.
- Maintainable.
- Independent.

2.17 Volume Summary

The **Backend Engineering Architecture** defines the complete internal engineering model for the BiteFixes backend.

This volume establishes:

- FastAPI architecture.
- Layer separation.
- Project structure.
- Service design.
- AI integration model.
- Database access strategy.
- Security foundations.
- Development standards.

The backend becomes the central foundation where all BiteFixes capabilities converge.

End of Volume 02

**BiteFixes Technical Implementation
Blueprint**

**BiteFixes Technical Implementation
Blueprint**

Volume 03

**Database Implementation Architecture
(DIA)**

Version: 1.0

Classification: Technical Implementation Documentation

Status: Draft

Table of Contents

3.1 Purpose

3.2 Database Architecture Vision

3.3 Database Technology Stack

3.4 Database Design Principles

3.5 Multi-Tenant Database Architecture

3.6 Core Entity Model

3.7 Customer Management Schema

3.8 Company and Tenant Schema

3.9 Service Management Schema

3.10 Ticket Management Schema

3.11 Conversation and Chat Memory Schema

3.12 Bitey AI Knowledge Schema

3.13 AI Intelligence Data Model

3.14 User and Permission Schema

3.15 Audit and Logging Schema

3.16 Database Security Model

3.17 Indexing Strategy

3.18 Database Backup Strategy

3.19 Migration Strategy

3.20 Volume Summary

3.1 Purpose

Overview

The Database Implementation Architecture defines the complete data foundation of BiteFixes.

The database is responsible for storing and managing:

- Business information.
- Customer information.
- Technical operations.
- AI knowledge.
- Conversation memory.
- SaaS tenant data.
- Security information.

Database Objective

The objective is:

Build a secure, scalable, intelligent data platform capable of supporting BiteFixes from startup stage to enterprise SaaS operation.

3.2 Database Architecture Vision

Overview

Supabase PostgreSQL acts as the central memory and operational database of BiteFixes.

Architecture:

Applications

|



BiteFixes Backend

|



Supabase Platform

|



PostgreSQL Database

|

├ Business Data

├ Customer Data

├ AI Memory

├ Knowledge

└ Operational Data

3.3 Database Technology Stack

Primary database:

PostgreSQL

Supabase

Row Level Security

Database Functions

Realtime Engine

Storage

Database Responsibilities

PostgreSQL

Responsible for:

- Structured data.
- Relationships.
- Transactions.
- Queries.

Supabase

Provides:

- Authentication.
- APIs.
- Storage.
- Realtime communication.

3.4 Database Design Principles

Principle 1 — Data Ownership

Every data record must have a clear owner.

Example:

Company

owns

Users

Customers

Tickets

Conversations

Principle 2 — Tenant Isolation

One company must never access another company's data.

Principle 3 — Historical Preservation

Operational history should not be deleted unnecessarily.

Examples:

- Conversations.
- Tickets.
- Repairs.
- AI decisions.

Principle 4 — AI Data Availability

Data should support:

- Learning.
- Analysis.
- Automation.

3.5 Multi-Tenant Database Architecture

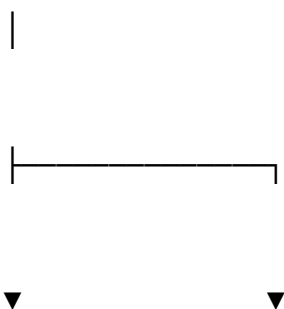
Overview

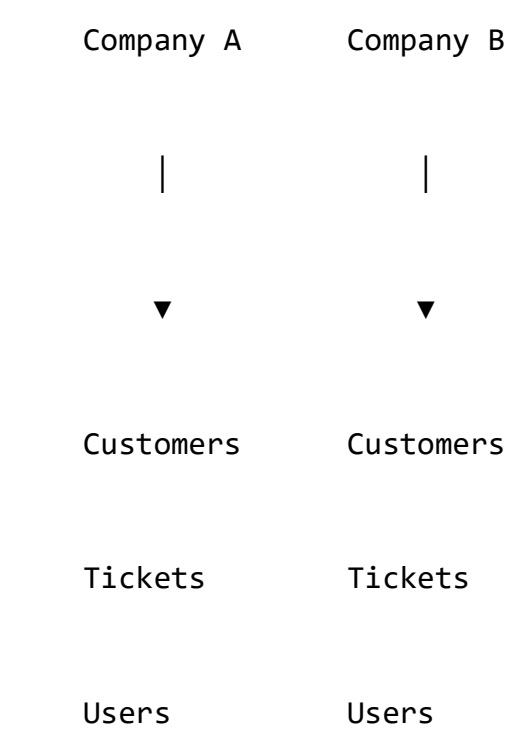
BiteFixes is designed as a SaaS platform.

Multiple companies operate inside the same platform.

Tenant Model

BiteFixes Platform





Tenant Identification

Main field:

`tenant_id`

Every business table must include:

`tenant_id` UUID

3.6 Core Entity Model

Main entities:

Tenant

Company

User

Customer

Device

Service

Ticket

Conversation

Message

Knowledge Article

AI Log

Audit Record

Relationship Overview

Company

|

└─ Users

└─ Customers

└─ Devices

└ Tickets

└ Conversations

Customer

|

└ Devices

└ Tickets

└ Messages

3.7 Customer Management Schema

Table

clientes

Purpose:

Stores customer information.

Fields:

id

tenant_id

name

email

phone

whatsapp

document

language

created_at

updated_at

Usage:

Used by:

- Support.
- Sales.
- AI personalization.

Customer Lifecycle

Created

↓

Active

↓

Service Usage

↓

Historical

↓

Archived

3.8 Company and Tenant Schema

Table

`empresas`

Purpose:

Represents SaaS customers.

Fields:

`id`

`name`

`business_type`

`plan`

`status`

`created_at`

Tenant Isolation

Each company receives:

- Unique tenant ID.
- Own users.
- Own customers.
- Own tickets.

3.9 Service Management Schema

Table

`servicios`

Purpose:

Stores technical services offered.

Fields:

`id`

`tenant_id`

`name`

`category`

`description`

`price`

active

created_at

Examples:

Laptop Repair

SSD Upgrade

Network Installation

CCTV Installation

Software Support

3.10 Ticket Management Schema

Table

tickets

Purpose:

Central operational workflow.

Fields:

id

tenant_id

customer_id

service_id

title

description

priority

status

assigned_user

created_at

updated_at

Ticket Status

OPEN



ASSIGNED



IN_PROGRESS



RESOLVED



CLOSED

3.11 Conversation and Chat Memory Schema

Table

conversaciones

Purpose:

Stores customer interactions.

Fields:

id

tenant_id

customer_id

channel

status

created_at

Messages Table

historial_chats

Fields:

id

conversation_id

sender

message

intent

ai_response

created_at

Memory Purpose

Used by Bitey for:

- Context.
- Personalization.
- History.

3.12 Bitey AI Knowledge Schema

Table

base_conhecimento

Purpose:

AI knowledge repository.

Fields:

id

tenant_id

title

category

question

answer

tags

intent

service_id

active

created_at

Knowledge Usage

Bitey uses:

Customer Question



Knowledge Search



Relevant Answer



Response

3.13 AI Intelligence Data Model

AI Logs Table

ia_logs

Purpose:

Track AI decisions.

Fields:

id

tenant_id

message

intent

confidence

response

model

created_at

AI Analytics

Supports:

- Accuracy measurement.
- Improvement.
- Training.

3.14 User and Permission Schema

Table

usuarios

Fields:

id

tenant_id

name

email

role

active

created_at

Roles

OWNER

ADMIN

TECHNICIAN

SUPPORT

CUSTOMER

3.15 Audit and Logging Schema

Table

audit_logs

Stores:

- User actions.
- Configuration changes.
- Security events.

Fields:

id

tenant_id

user_id

action

entity

timestamp

3.16 Database Security Model

Security mechanisms:

Authentication

+

Authorization

+

RLS Policies

+

Audit Logs

+

Encryption

Row Level Security

Example:

A company can only access:

```
WHERE tenant_id = current_user_tenant
```

3.17 Indexing Strategy

Important indexes:

```
tickets(customer_id)
```

```
tickets(status)
```

```
clientes(phone)
```

```
historial_chats(conversation_id)
```

```
base_conhecimento(intent)
```

Purpose:

Improve:

- Search.
- AI response speed.
- Reports.

3.18 Database Backup Strategy

Database protection:

Primary Database



Automatic Backup



Archive Storage



Recovery Environment

3.19 Migration Strategy

Database changes use controlled migrations.

Example:

Migration 001

Create Users

Migration 002

Create Customers

Migration 003

Create Tickets

Migration 004

Create AI Tables

Migration Rules

Every migration requires:

- Version number.
- Description.
- Testing.
- Rollback plan.

3.20 Volume Summary

The **Database Implementation Architecture** defines the complete data foundation of BiteFixes.

This volume establishes:

- PostgreSQL architecture.
- Supabase implementation.
- SaaS multi-tenancy.
- Customer management.

- Ticket workflow.
- AI memory.
- Knowledge base.
- Security model.
- Migration strategy.

The database becomes the permanent memory layer of BiteFixes and the foundation that enables Bitey AI intelligence.

End of Volume 03

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 04

Bitey AI Engine Implementation Architecture (BAEIA)

Version: 1.0

Classification: AI Technical Architecture Documentation

Status: Draft

Table of Contents

4.1 Purpose

4.2 Bitey AI Vision

4.3 AI Engine Architecture

4.4 AI Processing Pipeline

4.5 Message Understanding Layer

4.6 Intent Detection Engine

4.7 Knowledge Retrieval Engine

4.8 Conversation Memory Engine

4.9 Decision and Action Engine

4.10 Response Generation Architecture

4.11 AI Confidence System

4.12 Ticket Automation Engine

4.13 AI Logging and Analytics

4.14 External AI Provider Integration

4.15 AI Security Model

4.16 AI Improvement Lifecycle

4.17 Future Autonomous AI Architecture

4.18 Volume Summary

4.1 Purpose

Overview

The Bitey AI Engine Implementation Architecture defines the internal design of BiteFixes artificial intelligence system.

Bitey is the central intelligence layer responsible for:

- Understanding customer requests.
- Analyzing technical problems.
- Retrieving knowledge.
- Maintaining conversation context.
- Automating workflows.
- Supporting business decisions.

AI Objective

The objective is:

Create an intelligent service engine capable of understanding, assisting, and automating technical support operations.

4.2 Bitey AI Vision

Overview

Bitey is not designed as a simple chatbot.

Bitey operates as an AI orchestration layer.

Traditional Chatbot

User



Question



Answer

Bitey Architecture

User



Understanding



Context Analysis



Knowledge Retrieval



Decision Engine

↓

Action Execution

↓

Response

Bitey Responsibilities

Bitey manages:

- Conversation intelligence.
- Customer memory.
- Technical diagnosis.
- Service recommendation.
- Ticket creation.
- Workflow automation.

4.3 AI Engine Architecture

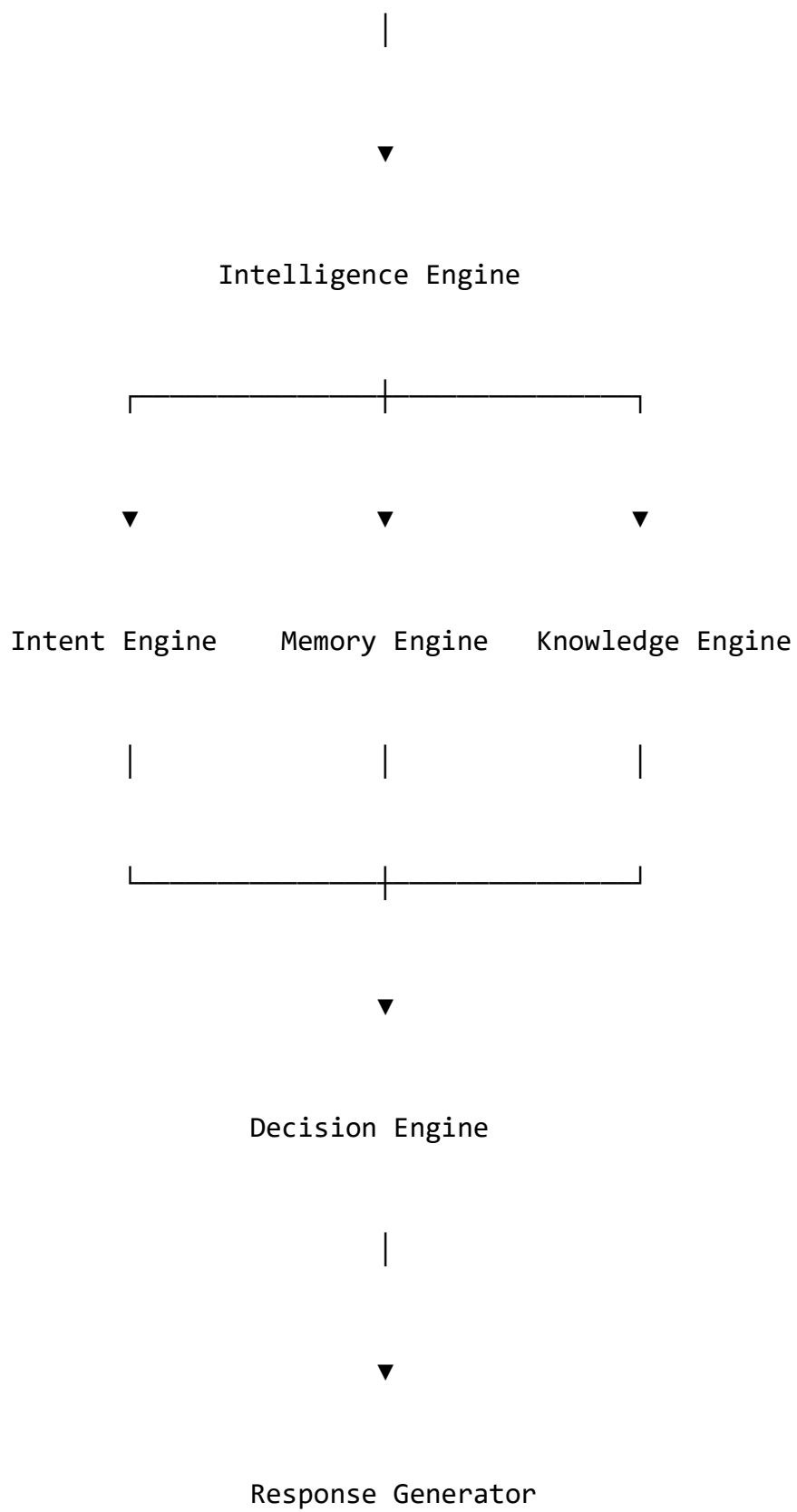
High-Level Architecture

Customer Message

|

▼

Message Processing Layer





4.4 AI Processing Pipeline

Overview

Every message follows a controlled processing pipeline.

Complete Flow

Message Received



Language Detection



Text Normalization



Customer Identification



Memory Retrieval



Intent Detection



Knowledge Search



Decision Processing



Response Generation



Action Execution



History Storage

4.5 Message Understanding Layer

Overview

The first AI layer transforms raw messages into structured information.

Input Example

Customer:

"Mi notebook está lento"

Processing

The system extracts:

```
{  
  "language": "es",  
  "topic": "computer",  
  "problem": "slow_performance",  
  "urgency": "normal"  
}
```

Functions

Responsible for:

- Language detection.
- Normalization.
- Keyword extraction.
- Context preparation.

4.6 Intent Detection Engine

Overview

Intent Detection identifies what the customer wants.

Example

Input:

"Quiero colocar más memoria RAM"

Detected:

Intent:

upgrade_hardware

Intent Architecture

Message

↓

Normalization

↓

Keyword Analysis

↓

Synonym Matching

↓

Weight Calculation

↓

Intent Selection

Intent Data Source

Table:

sinonimos_ia

Intent Example

Intent:

upgrade_hardware

Keywords:

RAM

memory

SSD

upgrade

Weight:

5

Confidence Score

Bitey calculates confidence:

0-3

Low confidence

4-7

Medium confidence

8-10

High confidence

4.7 Knowledge Retrieval Engine

Overview

The Knowledge Engine provides technical answers from the BiteFixes knowledge base.

Knowledge Flow

Customer Question



Intent Detection



Knowledge Search



Relevant Articles



Response Generation

Knowledge Sources

Primary:

base_conhecimento

Future:

- Technical manuals.
- Manufacturer documentation.
- Internal procedures.
- AI-generated knowledge.

Search Methods

Evolution:

Version 1

Keyword matching.

Version 2

Semantic search.

Version 3

Vector embeddings.

4.8 Conversation Memory Engine

Overview

Memory allows Bitey to understand previous interactions.

Memory Types

Short-Term Memory

Current conversation.

Example:

Customer messages today.

Long-Term Memory

Historical information.

Example:

Previous repairs.

Business Memory

Customer relationship.

Example:

Preferred service.

Memory Architecture

Customer

|



Conversation History

|



Memory Processing

|



Bitey Context

4.9 Decision and Action Engine

Overview

The Decision Engine determines what action should happen.

Decision Examples

Customer asks:

"How much does SSD upgrade cost?"

Possible actions:

Answer directly

OR

Create quotation request

OR

Create support ticket

Decision Model

Intent

+

Confidence

+

Customer Context

+

Business Rules

↓

Action

Possible Actions

- Send response.
- Create ticket.
- Assign technician.
- Request information.

- Escalate.

4.10 Response Generation Architecture

Overview

The Response Layer creates customer-facing answers.

Response Sources

Priority:

1. Knowledge Base
2. Business Rules
3. AI Generation
4. External AI Provider

Response Requirements

Responses must be:

- Clear.
- Accurate.
- Multilingual.
- Customer friendly.

4.11 AI Confidence System

Overview

Confidence determines automation level.

Automation Rules

High Confidence

Example:

95%

Action:

Automatic response.

Medium Confidence

Example:

70%

Action:

Response + verification.

Low Confidence

Example:

40%

Action:

Create ticket or request human support.

4.12 Ticket Automation Engine

Overview

Bitey automatically creates tickets when necessary.

Ticket Creation Flow

Customer Message



Problem Detection



No Knowledge Solution



Create Ticket



Assign Workflow



Notify Customer

Ticket Information Generated

Includes:

- Customer.
- Problem.
- Intent.
- Priority.
- AI analysis.
- Conversation history.

4.13 AI Logging and Analytics

Overview

Every AI decision is recorded.

AI Logs Store

Input

Intent

Confidence

Knowledge Used

Response

Action

Timestamp

Analytics Examples

Measure:

- Resolution rate.
- Automation percentage.
- Intent accuracy.
- Failed responses.

4.14 External AI Provider Integration

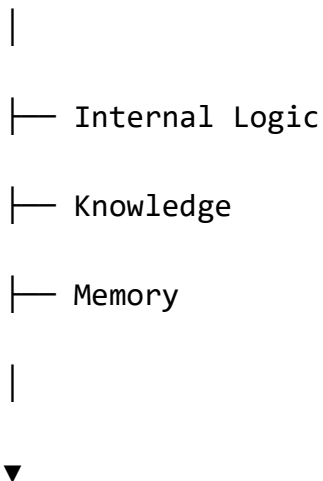
Overview

External AI models are enhancement layers.

They do not replace Bitey.

Architecture

Bitey Core



External AI Models

(OpenAI / Gemini)

External AI Usage

Examples:

- Complex reasoning.
- Natural language improvement.
- Summarization.
- Translation.

4.15 AI Security Model

Protection includes:

- Data privacy.
- Prompt protection.
- Access control.
- Audit logging.

AI Security Rules

Bitey must:

- Protect customer information.
- Avoid unauthorized actions.
- Record decisions.
- Validate external responses.

4.16 AI Improvement Lifecycle

Continuous Improvement

Conversation Data



Analysis



Knowledge Improvement



Intent Optimization



Better Responses

Improvement Sources

- Customer conversations.
- Technician feedback.
- Failed responses.
- New services.

4.17 Future Autonomous AI Architecture

Future Bitey evolution:

Observe



Understand



Predict



Recommend



Execute



Learn

Future Capabilities

- Autonomous diagnostics.
- Predictive maintenance.
- AI technician assistant.
- Business intelligence agent.
- Self-improving knowledge system.

4.18 Volume Summary

The **Bitey AI Engine Implementation Architecture** defines the intelligence foundation of BiteFixes.

This volume establishes:

- AI processing pipeline.
- Intent detection.
- Knowledge retrieval.
- Memory management.
- Decision automation.
- Ticket generation.
- AI analytics.
- External AI integration.
- Future autonomous evolution.

Bitey becomes the central intelligence system that transforms BiteFixes from a support application into an intelligent SaaS platform.

End of Volume 04

**BiteFixes Technical Implementation
Blueprint**

**BiteFixes Technical Implementation
Blueprint**

Volume 05

**API & Integration Implementation
Architecture (AIIA)**

Version: 1.0

Classification: Technical Implementation Documentation

Status: Draft

Table of Contents

5.1 Purpose

5.2 API Architecture Vision

5.3 API Design Principles

5.4 FastAPI API Structure

5.5 API Versioning Strategy

5.6 Authentication Architecture

5.7 Authorization Model

5.8 Customer API

5.9 Ticket API

5.10 Service API

5.11 Bitey AI Chat API

5.12 Knowledge API

5.13 Mobile Application API

5.14 WordPress Integration API

5.15 WhatsApp Integration Architecture

5.16 Webhook Architecture

5.17 External Partner Integrations

5.18 API Security

5.19 API Monitoring

5.20 Volume Summary

5.1 Purpose

Overview

The API & Integration Implementation Architecture defines how BiteFixes communicates internally and externally.

The API layer is the communication bridge between:

- Web applications.
- Mobile applications.
- WhatsApp.
- WordPress.
- External services.
- Enterprise partners.

API Objective

The objective is:

Provide a secure, scalable, and standardized communication layer that allows every BiteFixes component to interact with the platform.

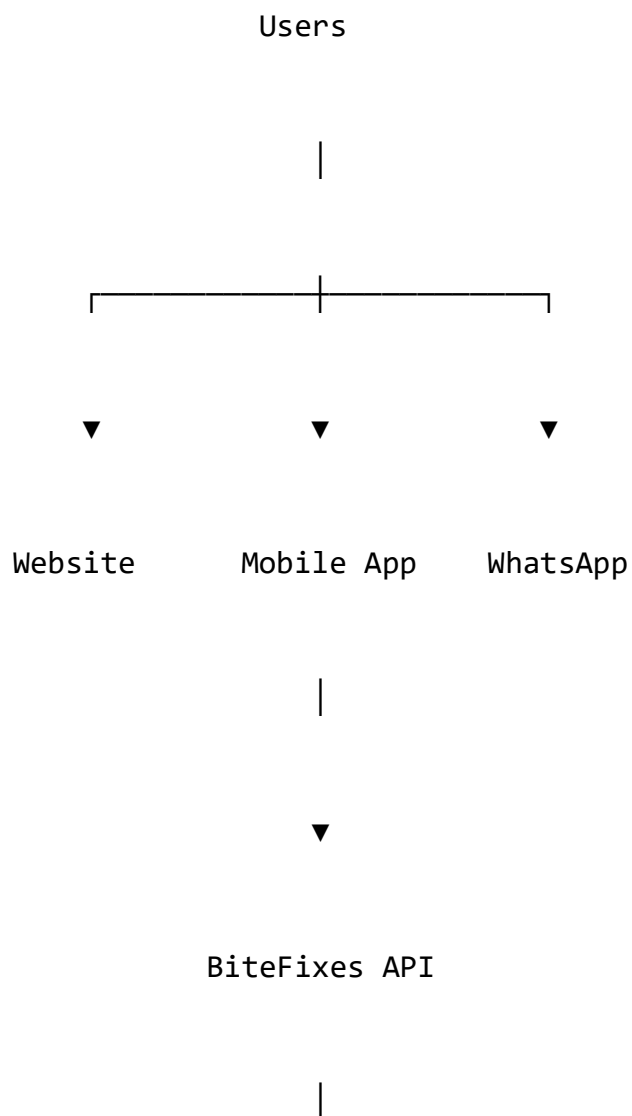
5.2 API Architecture Vision

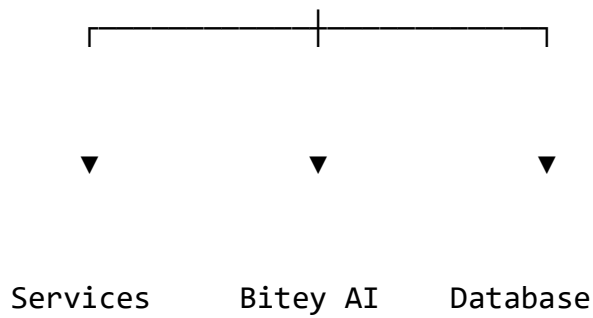
Overview

BiteFixes follows an API-first architecture.

Every major capability is exposed through APIs.

API Ecosystem





5.3 API Design Principles

Principle 1 — API First

New features are designed as APIs before interfaces.

Principle 2 — Stateless Communication

Requests contain all required information.

Benefits:

- Scalability.
- Reliability.
- Easier maintenance.

Principle 3 — Consistent Responses

All APIs follow standardized responses.

Example:

```
{  
  "success": true,  
  "data": {},  
  "message": "Operation completed"  
}
```

Principle 4 — Security by Default

Every API requires:

- Authentication.
- Authorization.
- Validation.

5.4 FastAPI API Structure

Production structure:

app/

└─ routers/

auth.py

clientes.py

tickets.py

servicios.py

chat.py

knowledge.py

integrations.py

Router Responsibility

Routers handle:

- HTTP requests.
- Validation.
- Response formatting.

Business logic remains in services.

5.5 API Versioning Strategy

Overview

BiteFixes APIs use version control.

Example:

/api/v1/

Version Evolution

Example:

v1

Initial SaaS API

v2

Advanced enterprise API

v3

AI autonomous API

Benefits

- Compatibility.
- Safer evolution.
- Easier integrations.

5.6 Authentication Architecture

Overview

Authentication verifies user identity.

Authentication Flow

User Login

↓

Credentials Validation

↓

Token Generation



API Access

Authentication Methods

Supported:

- Email/password.
- Supabase Auth.
- OAuth providers.
- API keys for integrations.

Token Model

Example:

```
{  
  "user_id": "123",  
  "tenant_id": "company001",  
  "role": "admin"  
}
```

5.7 Authorization Model

Overview

Authorization controls what users can do.

RBAC Model

Owner

|

Admin

|

Technician

|

Support

|

Customer

Permission Examples

Role	Permission
Owner	Full access
Admin	Company management
Technician	Assigned tickets
Support	Customer assistance
Customer	Own requests

5.8 Customer API

Purpose

Manages customer operations.

Endpoints

Create Customer

POST /api/v1/customers

Get Customer

GET /api/v1/customers/{id}

Update Customer

PUT /api/v1/customers/{id}

Example Response

```
{  
  "id":10,  
  "name":"Customer Name",  
  "phone":"+55xxxxxxxx"  
}
```

5.9 Ticket API

Purpose

Manages technical workflows.

Endpoints

Create ticket:

POST /api/v1/tickets

List tickets:

GET /api/v1/tickets

Update status:

PATCH /api/v1/tickets/{id}

Ticket Response

```
{  
  "id":100,  
  "status":"IN_PROGRESS",  
  "priority":"HIGH"  
}
```


5.10 Service API

Purpose

Manages technical services.

Endpoints:

GET /api/v1/services

POST /api/v1/services

Examples:

- Repair.
- Installation.
- Upgrade.
- Consulting.

5.11 Bitey AI Chat API

Overview

This is the central AI endpoint.

Endpoint

POST /api/v1/chat

Request

```
{  
  "message": "Meu notebook está lento",  
  "customer_id": 10,  
  "channel": "whatsapp"  
}
```

Processing

Request

↓

Bitey Engine

↓

Intent Detection

↓

Knowledge Search

↓

Response

Response

```
{  
  "response": "Podemos realizar upgrade de SSD e memória",  
}
```

```
"intent": "upgrade_hardware",  
"confidence": 5,  
"ticket_created": false  
}
```

5.12 Knowledge API

Purpose

Manages AI knowledge.

Endpoints:

Create knowledge:

POST /api/v1/knowledge

Search:

GET /api/v1/knowledge/search

Usage

Used by:

- Administrators.
- Bitey AI.
- Technicians.

5.13 Mobile Application API

Overview

Flutter applications communicate through the same API.

Mobile Features

Customer:

- Login.
- Chat.
- Tickets.
- Notifications.
- Service requests.

Technician:

- Assigned tickets.
- Customer information.
- Status updates.

Mobile Architecture

Flutter App

|



BiteFixes API

|



Backend Services

5.14 WordPress Integration API

Overview

The website acts as a customer acquisition channel.

Integration Features

WordPress can:

- Display services.
- Send support requests.
- Authenticate customers.
- Open chat.

Flow

Website



WordPress Plugin



BiteFixes API



Bitey AI

5.15 WhatsApp Integration Architecture

Overview

WhatsApp is a primary communication channel.

Architecture

Customer WhatsApp



WhatsApp Cloud API



Webhook



BiteFixes Backend



Bitey AI



Response

WhatsApp Capabilities

- Customer identification.
- Automated responses.
- Ticket creation.
- Notifications.

5.16 Webhook Architecture

Overview

Webhooks allow external systems to send events.

Webhook Flow

External Event



Webhook Endpoint



Validation



Processing



Action

Webhook Security

Protection:

- Signature validation.
- Token verification.
- Rate limiting.

5.17 External Partner Integrations

Future integrations:

- Payment providers.
- Hardware suppliers.
- ERP systems.
- CRM platforms.
- Monitoring tools.

Integration Model

Partner System



API Gateway



BiteFixes Services

5.18 API Security

Security controls:

- HTTPS.
- Authentication.
- Authorization.
- Input validation.
- Rate limiting.
- Logging.

5.19 API Monitoring

Metrics:

- Requests per second.
- Response time.
- Errors.
- Availability.
- Integration failures.

Monitoring Flow

API Request

↓

Log

↓

Metric

↓

Alert

↓

Action

5.20 Volume Summary

The **API & Integration Implementation Architecture** defines the communication foundation of BiteFixes.

This volume establishes:

- API-first strategy.
- FastAPI endpoints.
- Authentication.
- Authorization.
- Mobile communication.
- WordPress integration.
- WhatsApp integration.
- Webhooks.
- Enterprise integrations.
- API security.

The API layer becomes the universal communication interface connecting the entire BiteFixes ecosystem.

End of Volume 05

**BiteFixes Technical Implementation
Blueprint**

**BiteFixes Technical Implementation
Blueprint**

Volume 06

**Frontend & Mobile Implementation
Architecture (FMIA)**

Version: 1.0

Classification: Technical Implementation Documentation

Status: Draft

Table of Contents

6.1 Purpose

6.2 Frontend Architecture Vision

6.3 Channel Architecture Model

6.4 Web Platform Architecture

6.5 WordPress Integration Architecture

6.6 Customer Portal Architecture

6.7 Technician Portal Architecture

6.8 Flutter Mobile Application Architecture

6.9 Mobile Application Layers

6.10 State Management Architecture

6.11 Authentication Experience

6.12 Bitey AI User Interface

6.13 Notification Architecture

6.14 Multilingual User Experience

6.15 Frontend Security

6.16 User Experience Standards

6.17 Frontend Development Workflow

6.18 Volume Summary

6.1 Purpose

Overview

The Frontend & Mobile Implementation Architecture defines the user interaction layer of BiteFixes.

This layer provides access to:

- Customers.
- Technicians.
- Business administrators.
- Partners.

Objective

The objective is:

Create a unified, multilingual, and intelligent user experience connected to Bitey AI.

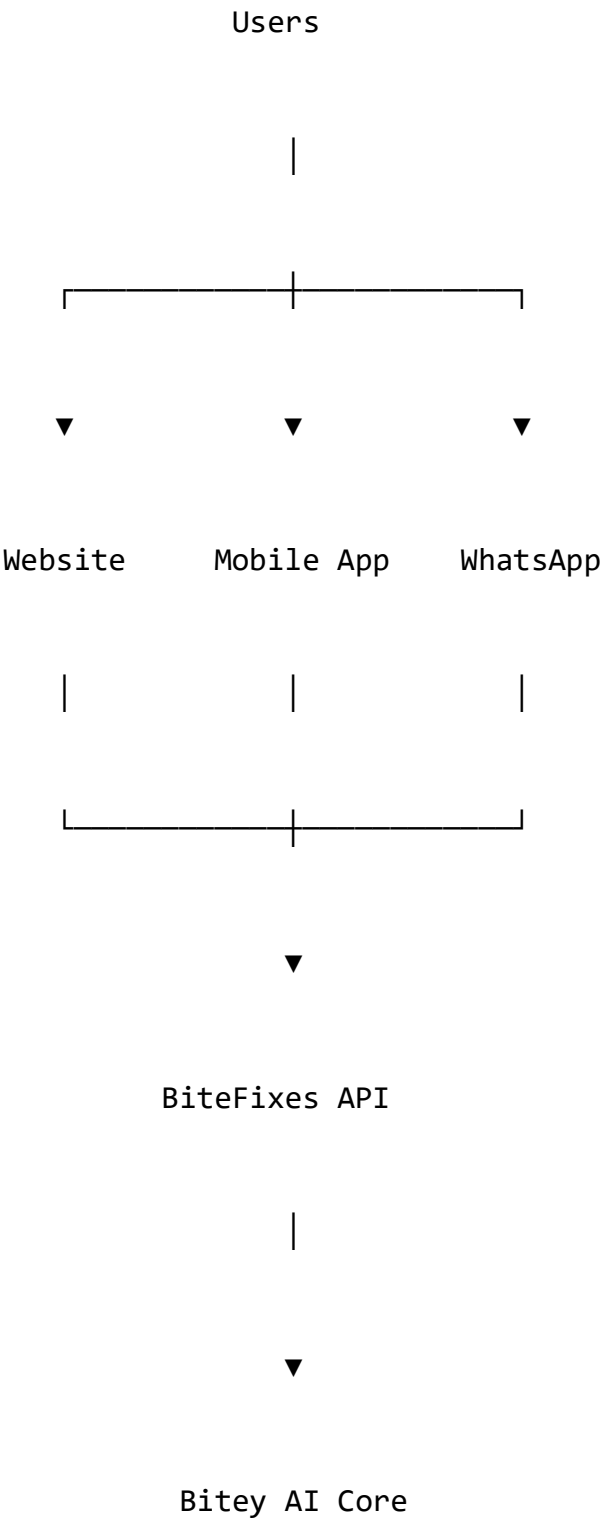
6.2 Frontend Architecture Vision

Overview

BiteFixes follows a multi-channel frontend strategy.

All interfaces consume the same backend services.

Frontend Ecosystem



Frontend Principles

Consistency

All channels provide the same business experience.

Simplicity

Users should complete tasks with minimum complexity.

Intelligence

Bitey assists users throughout the experience.

Accessibility

Interfaces must support different user capabilities.

6.3 Channel Architecture Model

BiteFixes supports four main channels:

Channel 1 — Public Website

Purpose:

Customer acquisition.

Features:

- Services.
- Company information.
- Contact.
- AI chat.

Channel 2 — Customer Portal

Purpose:

Existing customer management.

Features:

- Tickets.
- Conversations.
- Service history.
- Documents.

Channel 3 — Technician Portal

Purpose:

Technical operations.

Features:

- Assigned jobs.
- Customer data.
- Repair workflow.

Channel 4 — Mobile Application

Purpose:

Continuous access.

Features:

- Chat.
- Notifications.
- Requests.
- Tracking.

6.4 Web Platform Architecture

Overview

The web platform is the main public entry point.

Technology Options

Initial:

WordPress

+

Custom Integration

+

BiteFixes API

Future:

React / Next.js Application

Web Architecture

Browser



Frontend Interface



BiteFixes API



Backend Services



Database

Website Responsibilities

- Marketing.
- Customer registration.
- Service requests.
- AI assistance.

6.5 WordPress Integration Architecture

Overview

WordPress operates as the initial business website.

Integration Components

WordPress

|

└─ BiteFixes Plugin

|

└─ API Connector

|

└─ Chat Widget

Features

Floating Bitey Chat

Provides:

- Customer assistance.
- Service information.
- Lead generation.

Request Budget Section

Allows:

- Service request.
- Customer information.
- Automatic ticket creation.

6.6 Customer Portal Architecture

Purpose

Provides customers with self-service capabilities.

Customer Dashboard

Contains:

Profile

Tickets

Conversations

Services

Invoices

Notifications

Customer Workflow

Customer Login



Dashboard



Request Service



Bitey Assistance



Ticket Tracking

6.7 Technician Portal Architecture

Purpose

Allows technicians to manage operations.

Technician Dashboard

Features:

- Assigned tickets.
- Customer information.

- Technical notes.
- Status updates.

Workflow

Ticket Created



Assignment



Diagnosis



Repair



Completion

6.8 Flutter Mobile Application Architecture

Overview

Flutter provides the cross-platform mobile experience.

Supported:

- Android.
- iOS.

Mobile Architecture

Flutter UI

|



Application Logic

|



API Client

|



BiteFixes Backend

Mobile Features

Customer:

- Account.
- Chat.
- Tickets.
- Notifications.

Technician:

- Work orders.
- Status updates.
- Communication.

6.9 Mobile Application Layers

Flutter follows layered architecture.

Presentation Layer

Contains:

- Screens.
- Widgets.
- UI components.

Application Layer

Contains:

- State management.
- Business flows.

Domain Layer

Contains:

- Entities.
- Business rules.

Data Layer

Contains:

- API communication.
- Local storage.

Structure Example

lib/

├─ presentation/

├─ application/

├─ domain/

├─ data/

├─ services/

└─ models/

6.10 State Management Architecture

Purpose

Controls application state.

Possible technologies:

- Riverpod.
- Provider.
- Bloc.

State Example

User State

Ticket State

Chat State

Notification State

6.11 Authentication Experience

Overview

Authentication must be consistent across channels.

Login Flow

User



Credentials

↓

Authentication Service

↓

Token

↓

Application Access

Authentication Features

- Login.
- Registration.
- Password recovery.
- Session management.

6.12 Bitey AI User Interface

Overview

Bitey has a dedicated conversational experience.

Chat Interface

Components:

Messages

Suggestions

Attachments

Service Options

Ticket Status

Intelligent Features

Bitey can show:

- Suggested actions.
- Service recommendations.
- Status updates.

6.13 Notification Architecture

Overview

Users receive real-time updates.

Notification Channels

Push Notification

Email

WhatsApp

SMS

In-App

Events

Examples:

- Ticket created.
- Technician assigned.
- Repair completed.

6.14 Multilingual User Experience

Overview

BiteFixes supports:

Portuguese

Spanish

English

Localization Components

Includes:

- Interface text.
- AI responses.
- Notifications.
- Documentation.

Language Detection

Bitey can identify:

User Language



Response Language

6.15 Frontend Security

Security controls:

- Secure authentication.
- Token protection.
- Input validation.
- Session management.

Mobile Security

Includes:

- Secure storage.
- Certificate validation.
- Application protection.

6.16 User Experience Standards

Requirements:

- Fast response.
- Simple navigation.
- Clear information.
- Consistent design.

UX Principles

Reduce Complexity

Users should not need technical knowledge.

Guide Users

Bitey assists decisions.

Transparency

Users see:

- Status.
- Progress.
- Actions.

6.17 Frontend Development Workflow

Process:

Design



Development



API Integration



Testing



Deployment



Improvement

6.18 Volume Summary

The **Frontend & Mobile Implementation Architecture** defines the complete user interaction layer of BiteFixes.

This volume establishes:

- Website architecture.
- WordPress integration.
- Customer portal.

- Technician portal.
- Flutter application.
- UI architecture.
- Authentication.
- Notifications.
- Multilingual experience.
- UX standards.

The frontend layer transforms BiteFixes technical capabilities into a simple, intelligent, and accessible user experience.

End of Volume 06

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 07

Infrastructure & Cloud Implementation Architecture (ICIA)

Version: 1.0

Classification: Infrastructure Technical Documentation

Status: Draft

Table of Contents

7.1 Purpose

7.2 Infrastructure Vision

7.3 Cloud Architecture Model

7.4 Environment Architecture

7.5 Development Environment

7.6 Staging Environment

7.7 Production Environment

7.8 Backend Hosting Architecture

7.9 Supabase Infrastructure Architecture

7.10 Container Architecture

7.11 Networking Architecture

7.12 Storage Architecture

7.13 Configuration Management

7.14 Deployment Architecture

7.15 CI/CD Infrastructure

7.16 Backup Architecture

7.17 Scalability Strategy

7.18 High Availability Model

7.19 Infrastructure Security

7.20 Volume Summary

7.1 Purpose

Overview

The Infrastructure & Cloud Implementation Architecture defines the physical and virtual environment required to operate BiteFixes.

This volume transforms the logical architecture into a production-ready infrastructure model.

Infrastructure Objective

The objective is:

Provide a secure, scalable, reliable cloud foundation capable of supporting BiteFixes from MVP stage to enterprise SaaS operation.

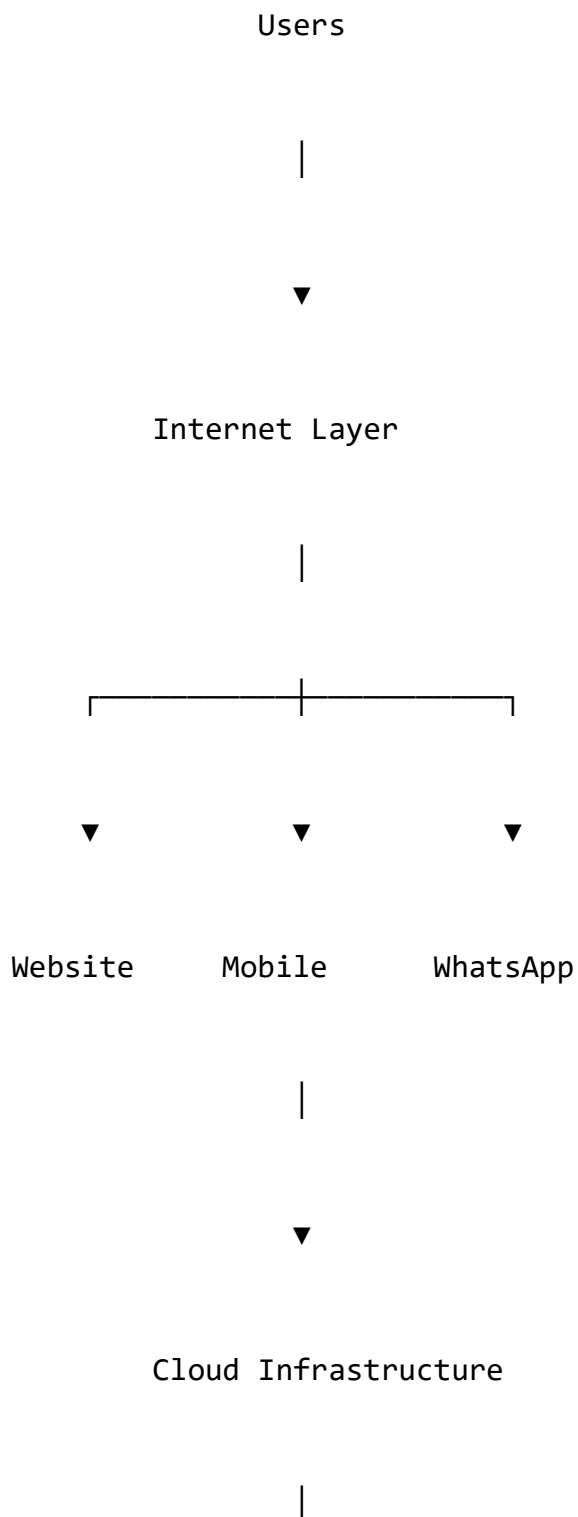
7.2 Infrastructure Vision

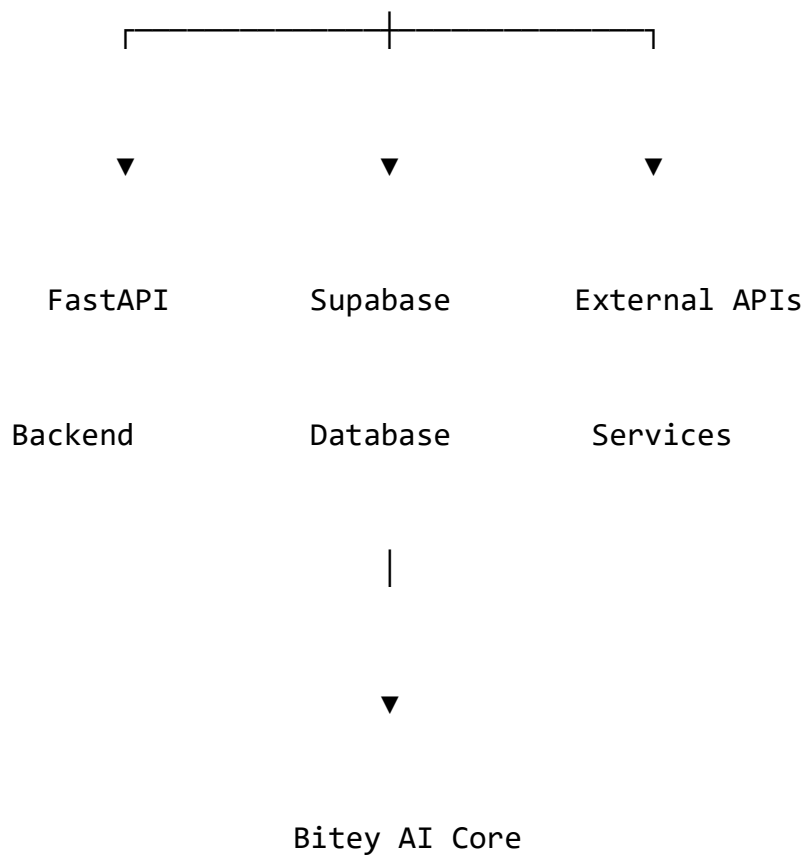
Overview

BiteFixes follows a cloud-first architecture.

The platform avoids dependency on physical servers and uses managed cloud services.

Infrastructure Model





7.3 Cloud Architecture Model

Initial Cloud Strategy

BiteFixes uses a managed cloud approach.

Primary components:

Application Hosting

+

Database Platform

+

Storage

+

Monitoring

+

External Integrations

Cloud Components

Application Layer

Technology:

- FastAPI.
- Python.
- Uvicorn.
- Docker.

Hosting:

- Render initially.

Data Layer

Technology:

- Supabase.
- PostgreSQL.

Communication Layer

Services:

- WhatsApp Cloud API.
- Email provider.
- Push notifications.

7.4 Environment Architecture

BiteFixes maintains three environments.

Environment Model

Development

↓

Staging

↓

Production

Purpose

Development

Used by developers.

Contains:

- Local code.
- Test database.
- Debug configuration.

Staging

Used before production.

Contains:

- Production-like environment.
- Integration testing.
- Validation.

Production

Real customer environment.

Contains:

- Live data.
- Security controls.
- Monitoring.

7.5 Development Environment

Local Architecture

Developer machine:

Windows/Linux/macOS

|



Python Environment

|



FastAPI

|



Local Configuration

|



Supabase Development Project

Development Requirements

Software:

Python 3.x

Git

Virtual Environment

Docker

IDE

Postman

7.6 Staging Environment

Purpose

Validation before release.

Staging Architecture

Git Repository

|



Deployment Pipeline

|



Staging Backend

|



Staging Database

Staging Tests

Includes:

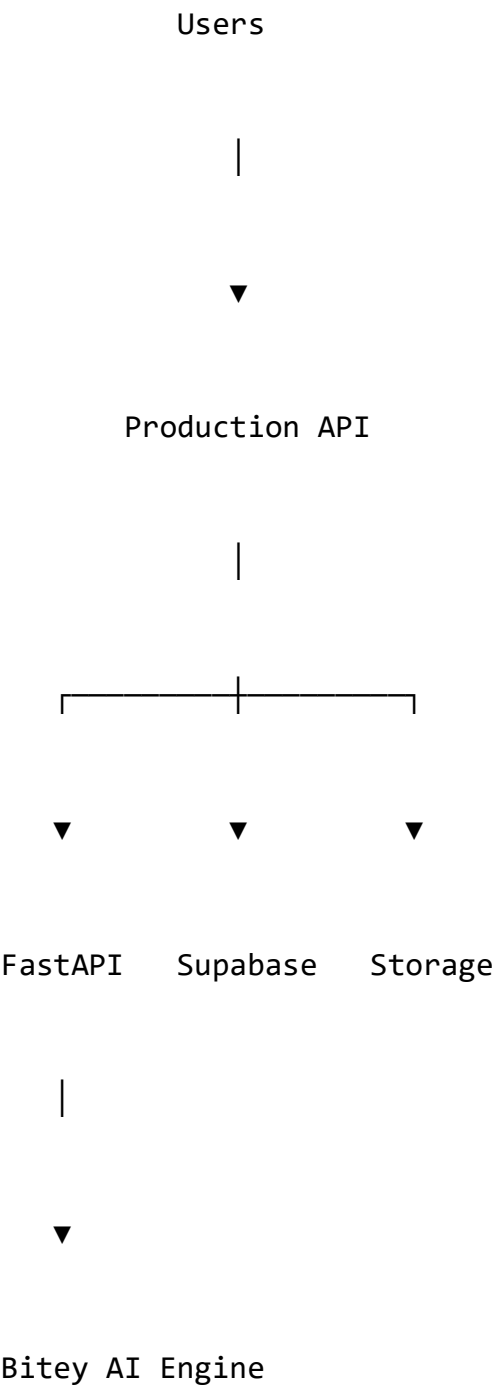
- API testing.
- Integration testing.
- AI response testing.
- Security verification.

7.7 Production Environment

Overview

Production hosts the live BiteFixes platform.

Production Architecture



Production Requirements

Must provide:

- Reliability.
- Monitoring.
- Backup.
- Security.
- Scalability.

7.8 Backend Hosting Architecture

Initial Platform

Recommended:

Render

|



Docker Container

|



FastAPI Application

Backend Runtime

Components:

Python Runtime

Uvicorn Server

Environment Variables

Application Logs

Health Checks

Health Endpoint

Example:

/health

Purpose:

- Availability monitoring.
- Deployment validation.

7.9 Supabase Infrastructure Architecture

Overview

Supabase is the central data platform.

Supabase Components

PostgreSQL

+

Authentication

+

Storage

+

Realtime

+

API Layer

Database Responsibilities

Stores:

- Customers.
- Companies.
- Tickets.
- Conversations.
- AI memory.
- Knowledge.

Storage Responsibilities

Stores:

- Images.
- Documents.
- Attachments.
- Repair evidence.

7.10 Container Architecture

Overview

BiteFixes uses containerization for portability.

Docker Model

Docker Image

|



Container

|



FastAPI Application

Benefits

- Same environment everywhere.
- Easier deployment.
- Better scalability.

7.11 Networking Architecture

Overview

Network communication follows controlled paths.

Network Flow

Client

↓

HTTPS

↓

API Gateway

↓

Backend



Database

Network Requirements

- HTTPS only.
- Secure connections.
- Protected database access.

7.12 Storage Architecture

Storage Types

Database Storage

For:

- Structured information.

Object Storage

For:

- Images.
- Files.

- Documents.

Backup Storage

For:

- Recovery copies.

7.13 Configuration Management

Overview

Configuration is externalized.

Environment Variables

Example:

DATABASE_URL

SUPABASE_URL

SUPABASE_KEY

SECRET_KEY

AI_PROVIDER_KEY

WHATSAPP_TOKEN

Rules

Never store secrets:

- Inside code.
- Inside repositories.
- Inside documentation.

7.14 Deployment Architecture

Deployment flow:

Developer

↓

Git Commit

↓

Repository

↓

CI/CD Pipeline

↓

Production Deployment

Deployment Strategy

Initial:

- Manual approval.

Future:

- Automated deployment.

7.15 CI/CD Infrastructure

Pipeline

Code Push

↓

Tests

↓

Security Checks

↓

Build

↓

Deploy

↓

Monitor

Pipeline Validation

Checks:

- Syntax.
- Tests.
- Dependencies.
- Security.

7.16 Backup Architecture

Objective

Protect business information.

Backup Model

Production Database



Automatic Backup



Backup Storage



Recovery Process

Backup Types

- Database backups.
- File backups.
- Configuration backups.

7.17 Scalability Strategy

Initial Scaling

Vertical:

- More CPU.
- More RAM.

Advanced Scaling

Horizontal:

Instance 1

Instance 2

Instance 3



Load Balancer

Future Evolution

Possible:

- Kubernetes.
- Microservices.
- Multi-region deployment.

7.18 High Availability Model

Availability strategy:

Monitoring

+

Backups

+

Automatic Recovery

+

Redundant Services

7.19 Infrastructure Security

Security controls:

- HTTPS.
- Secret management.
- Firewall rules.

- Access control.
- Monitoring.

Infrastructure Security Principle

Only authorized components communicate.

7.20 Volume Summary

The **Infrastructure & Cloud Implementation Architecture** defines the production foundation of BiteFixes.

This volume establishes:

- Cloud strategy.
- Environment separation.
- Render deployment model.
- Supabase infrastructure.
- Docker architecture.
- Networking.
- Storage.
- CI/CD.
- Backup.
- Scalability.
- Availability.

The infrastructure becomes the operational foundation that allows BiteFixes and Bitey AI to evolve from MVP into an enterprise SaaS platform.

End of Volume 07

**BiteFixes Technical Implementation
Blueprint**

**BiteFixes Technical Implementation
Blueprint**

Volume 08

**DevSecOps Implementation Architecture
(DSIA)**

Version: 1.0

Classification: Engineering Operations Documentation

Status: Draft

Table of Contents

8.1 Purpose

8.2 DevSecOps Vision

8.3 Development Lifecycle Model

8.4 Source Code Management Architecture

8.5 Git Repository Strategy

8.6 Branch Management Model

8.7 Code Review Process

8.8 Development Standards

8.9 Continuous Integration Architecture

8.10 Continuous Deployment Architecture

8.11 Automated Testing Pipeline

8.12 Security Integration

8.13 Dependency Management

8.14 Release Management

8.15 Environment Promotion Strategy

8.16 Infrastructure as Code

8.17 DevSecOps Monitoring

8.18 Engineering Governance

8.19 Volume Summary

8.1 Purpose

Overview

The DevSecOps Implementation Architecture defines the engineering processes required to develop, secure, test, and deploy BiteFixes.

The objective is to combine:

- Development.
- Operations.
- Security.

into one continuous lifecycle.

DevSecOps Objective

The goal is:

Deliver reliable, secure, and continuously improving BiteFixes software through automated engineering processes.

8.2 DevSecOps Vision

Overview

Traditional model:

Development



Operations



Security

BiteFixes Model

Development



Operations



Security



Continuous Delivery

DevSecOps Principles

Automation First

Reduce manual processes.

Security Early

Security starts during development.

Continuous Improvement

Every release improves the platform.

8.3 Development Lifecycle Model

BiteFixes follows:

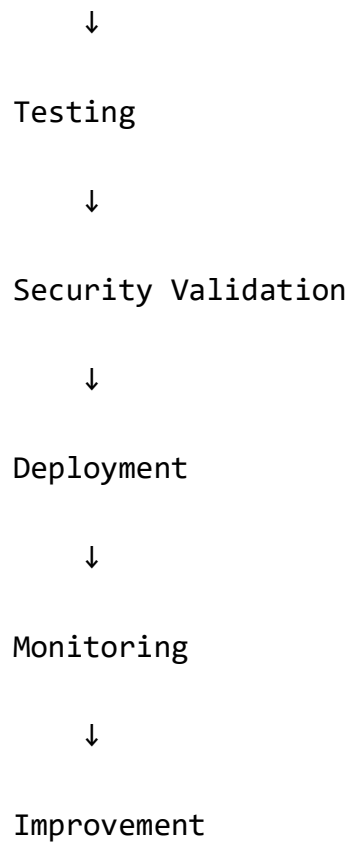
Planning



Development



Code Review



8.4 Source Code Management Architecture

Overview

Git is the source control system.

Repository:

GitHub

↓

BiteFixes Source Code

Repository Responsibilities

Stores:

- Backend.
- Frontend.
- Mobile.
- Infrastructure.
- Documentation.

Repository Structure

bitefixes-platform/

|— backend/

|— frontend/

|— mobile/

|— infrastructure/

|— documentation/

|— tests/

8.5 Git Repository Strategy

Main Branches

main

↓

production

develop

↓

integration

feature/*

↓

new development

Branch Purpose

Main

Contains stable production code.

Develop

Contains integrated changes.

Feature Branches

Used for new functionality.

Example:

feature/chat-improvement

feature/whatsapp-api

feature/ticket-system

8.6 Branch Management Model

Workflow

Developer



Feature Branch



Pull Request



Review

↓

Testing

↓

Merge

Branch Rules

Direct commits to main:

Not allowed.

Required:

- Pull Request.
- Review.
- Tests passing.

8.7 Code Review Process

Purpose

Maintain software quality.

Review Checklist

Before merging:

- Code quality.
- Security.
- Tests.
- Documentation.
- Performance.

Review Questions

Examples:

- Is the code maintainable?
- Is the architecture respected?
- Are errors handled?
- Are secrets protected?

8.8 Development Standards

Backend Standards

Language:

Python

Rules:

- PEP8 compliance.
- Type hints.

- Documentation.
- Testing.

Naming Convention

Python:

`create_ticket()`

`customer_service`

Frontend Standards

Requirements:

- Component organization.
- Reusable components.
- Consistent design.

Mobile Standards

Requirements:

- Clean architecture.
- Separation of layers.
- Automated tests.

8.9 Continuous Integration Architecture

Overview

Every code change is automatically validated.

CI Pipeline

Code Push



Build



Tests



Security Scan



Validation

CI Tools

Possible:

- GitHub Actions.
- Automated scripts.

- Quality scanners.

8.10 Continuous Deployment Architecture

Overview

Deployment moves validated code into environments.

CD Flow

Approved Code



Build Artifact



Deploy Staging



Validation



Deploy Production

8.11 Automated Testing Pipeline

Testing layers:

Unit Testing

Tests:

- Functions.
- Services.
- Business logic.

Integration Testing

Tests:

- APIs.
- Database.
- External services.

End-to-End Testing

Tests:

- Complete user scenarios.

AI Testing

Specific for Bitey:

Tests:

- Intent accuracy.
- Response quality.
- Knowledge retrieval.

8.12 Security Integration

Overview

Security is integrated into the pipeline.

Security Checks

Includes:

- Dependency scanning.
- Secret detection.
- Vulnerability analysis.
- Code analysis.

Security Flow

Code

↓

Security Scan

↓

Approval

↓

Deployment

8.13 Dependency Management

Purpose

Control external libraries.

Dependency Rules

Every dependency must have:

- Version control.
- Security evaluation.
- Documentation.

Python Example

`requirements.txt`

or

`pyproject.toml`

8.14 Release Management

Overview

Releases follow controlled versions.

Version Format

Example:

1.0.0

Major.Minor.Patch

Release Types

Major

Large architecture changes.

Minor

New features.

Patch

Bug fixes.

8.15 Environment Promotion Strategy

Code promotion:

Development

↓

Staging

↓

Production

Promotion Requirements

Before production:

- Tests completed.
- Security approved.
- Backup verified.

8.16 Infrastructure as Code

Overview

Infrastructure should be reproducible.

Managed Components

Future:

- Docker.
- Terraform.
- Cloud configuration.

Benefits

- Repeatability.
- Automation.
- Disaster recovery.

8.17 DevSecOps Monitoring

Monitoring includes:

Development Metrics

- Build failures.
- Deployment frequency.
- Test results.

Operational Metrics

- Availability.
- Errors.
- Performance.

8.18 Engineering Governance

Purpose

Maintain technical discipline.

Governance Rules

Every change requires:

- Documentation.
- Testing.
- Review.

Architecture Review

Important changes require:

- Technical evaluation.
- Security review.
- Impact analysis.

8.19 Volume Summary

The **DevSecOps Implementation Architecture** establishes the professional engineering workflow required to build BiteFixes as a scalable SaaS company.

This volume defines:

- Git strategy.
- Branch management.
- Code review.

- CI/CD.
- Testing.
- Security integration.
- Release management.
- Engineering governance.

DevSecOps becomes the mechanism that transforms BiteFixes development from individual coding into an enterprise software engineering process.

End of Volume 08

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 09

Monitoring & Observability Architecture (MOA)

Version: 1.0

Classification: Operations & Reliability Documentation

Status: Draft

Table of Contents

9.1 Purpose

9.2 Observability Vision

9.3 Monitoring Architecture Model

9.4 Logging Architecture

9.5 Application Performance Monitoring

9.6 Infrastructure Monitoring

9.7 Database Monitoring

9.8 API Monitoring

9.9 Bitey AI Monitoring

9.10 Business Metrics Monitoring

9.11 Alert Management Architecture

9.12 Dashboard Architecture

9.13 Incident Detection Process

9.14 Performance Optimization

9.15 Availability Monitoring

9.16 Capacity Planning

9.17 Observability Security

9.18 Future Intelligence Monitoring

9.19 Volume Summary

9.1 Purpose

Overview

The Monitoring & Observability Architecture defines the systems required to understand the health, performance, and behavior of BiteFixes.

Monitoring allows the platform team to answer:

- Is the system working?
- Is it fast enough?
- Are customers affected?
- Is Bitey AI performing correctly?
- Where is a problem occurring?

Objective

The objective is:

Create complete visibility over the BiteFixes platform, from infrastructure to customer experience.

9.2 Observability Vision

Traditional Monitoring

Traditional monitoring asks:

"Is the server online?"

Modern Observability

Observability asks:

"Why is the customer experiencing this problem?"

BiteFixes Model

Infrastructure

+

Application

+

Database

+

AI Engine

+

Business Data



Complete Visibility

9.3 Monitoring Architecture Model

Overview

The observability platform collects information from all components.

Architecture

Users



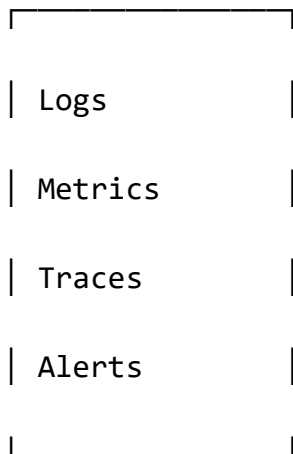
Applications



FastAPI Backend



Monitoring Layer



Dashboards



Operations Team

9.4 Logging Architecture

Purpose

Logs record system events.

Log Sources

Backend Logs

Includes:

- Requests.
- Errors.
- Exceptions.
- Performance.

AI Logs

Includes:

- User message.
- Detected intent.
- Confidence.
- Generated response.

Security Logs

Includes:

- Authentication attempts.
- Access events.
- Configuration changes.

Log Structure

Example:

```
{  
  "time": "2026-07-24",
```

```
"service": "bitey-engine",  
"level": "INFO",  
"event": "intent_detected",  
"intent": "upgrade_hardware"  
}
```

Log Levels

DEBUG

INFO

WARNING

ERROR

CRITICAL

9.5 Application Performance Monitoring

Overview

Measures application behavior.

Metrics

Track:

- Response time.
- Request volume.

- Errors.
- CPU usage.
- Memory usage.

FastAPI Monitoring

Important endpoints:

/

health

/chat

/tickets

/customers

Performance Example

Average API Response

< 500ms

Target

9.6 Infrastructure Monitoring

Components

Monitor:

- Render services.
- Containers.
- Network.
- Storage.

Infrastructure Metrics

Includes:

CPU

Detects:

- Resource saturation.

Memory

Detects:

- Memory leaks.

Disk

Detects:

- Storage problems.

Network

Detects:

- Connectivity issues.

9.7 Database Monitoring

Overview

Supabase/PostgreSQL monitoring is critical.

Database Metrics

Monitor:

- Connections.
- Query performance.
- Storage usage.
- Slow queries.

Database Health

Healthy

↓

Warning

↓

Critical

Database Optimization

Actions:

- Index improvement.
- Query optimization.
- Data archiving.

9.8 API Monitoring

Overview

APIs are the communication center of BiteFixes.

API Metrics

Measure:

- Requests per minute.
- Response codes.
- Latency.
- Failures.

HTTP Monitoring

Important statuses:

200

Successful

400

Client Error

500

Server Error

API Availability Target

Example:

99.9%

Service Availability Goal

9.9 Bitey AI Monitoring

Overview

Bitey requires specialized monitoring.

AI Metrics

Track:

Intent Accuracy

Example:

Did Bitey correctly identify:

upgrade_hardware

Confidence Distribution

Measure:

- High confidence responses.
- Low confidence failures.

Knowledge Performance

Measure:

- Articles used.
- Successful answers.

Escalation Rate

Measure:

How many conversations require human support.

AI Monitoring Flow

Customer Message



Intent Detection



Confidence



Response



Result Analysis

9.10 Business Metrics Monitoring

Overview

Technical monitoring is not enough.

BiteFixes also monitors business behavior.

Business Metrics

Includes:

- New customers.
- Tickets created.
- Resolution time.
- Conversion rate.
- Service demand.

Example Dashboard

Customers Today

Tickets Today

AI Resolutions

Pending Repairs

Revenue Impact

9.11 Alert Management Architecture

Purpose

Notify when action is required.

Alert Flow

Problem Detected



Rule Evaluation



Alert Created



Notification



Response Action

Alert Channels

Possible:

- Email.
- WhatsApp.
- Slack.
- Mobile notification.

Alert Examples

Critical

Backend unavailable.

Warning

Database storage increasing.

Information

New deployment completed.

9.12 Dashboard Architecture

Overview

Dashboards provide operational visibility.

Dashboard Types

Technical Dashboard

Shows:

- Servers.
- APIs.
- Errors.

AI Dashboard

Shows:

- Intent accuracy.

- Conversations.
- Automation.

Business Dashboard

Shows:

- Customers.
- Revenue.
- Services.

9.13 Incident Detection Process

Overview

Problems follow a structured process.

Incident Flow

Detection

↓

Analysis

↓

Resolution

↓

Documentation



Improvement

Incident Categories

Infrastructure

Example:

Server unavailable.

Application

Example:

API error.

Data

Example:

Database issue.

AI

Example:

Incorrect response.

9.14 Performance Optimization

Optimization areas:

- API speed.
- Database queries.
- AI response time.
- Frontend loading.

Optimization Cycle

Measure



Analyze



Improve



Measure Again

9.15 Availability Monitoring

Goal

Maintain reliable service.

Availability Components

Monitoring

+

Backups

+

Recovery

+

Alerts

9.16 Capacity Planning

Purpose

Prepare for growth.

Capacity Metrics

Monitor:

- Users.
- Requests.
- Database size.
- AI usage.

Growth Model

Startup

↓

SMB SaaS

↓

Enterprise

9.17 Observability Security

Protection:

- Access-controlled dashboards.
- Protected logs.
- Sensitive data masking.

Security Rule

Monitoring data is also protected data.

9.18 Future Intelligence Monitoring

Future evolution:

Bitey monitors itself.

AI Operations Model

Detect



Analyze



Predict



Recommend Action

Future Capabilities

- Predict failures.
- Recommend optimization.
- Detect abnormal behavior.

9.19 Volume Summary

The **Monitoring & Observability Architecture** establishes complete visibility of BiteFixes.

This volume defines:

- Logging.

- Metrics.
- Tracing.
- API monitoring.
- Database monitoring.
- Bitey AI monitoring.
- Business dashboards.
- Alerts.
- Incident management.
- Performance optimization.

Observability becomes the operational intelligence layer that allows BiteFixes to maintain reliability while scaling into an enterprise SaaS platform.

End of Volume 09

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 10

Security Implementation Architecture (SIA)

Version: 1.0

Classification: Security Architecture Documentation

Status: Draft

Table of Contents

10.1 Purpose

10.2 Security Vision

10.3 Security Architecture Model

10.4 Security Principles

10.5 Identity and Authentication Architecture

10.6 Authorization and Access Control

10.7 Multi-Tenant Security Model

10.8 API Security Architecture

10.9 Database Security Architecture

10.10 Data Protection Architecture

10.11 Encryption Strategy

10.12 Secret Management

10.13 Application Security

10.14 Mobile Application Security

10.15 WordPress Security

10.16 Bitey AI Security Architecture

10.17 Security Monitoring

10.18 Incident Response

10.19 Security Compliance Strategy

10.20 Volume Summary

10.1 Purpose

Overview

The Security Implementation Architecture defines the security framework protecting the BiteFixes ecosystem.

Security covers:

- Customer data.
- Business data.
- AI memory.
- Conversations.
- Applications.
- Infrastructure.

Security Objective

The objective is:

Protect the confidentiality, integrity, and availability of BiteFixes services while allowing secure growth as a SaaS platform.

10.2 Security Vision

Overview

Security is not an additional component.

Security is integrated into every layer.

Security Model

Users

↓

Authentication

↓

Authorization

↓

Application Security

↓

Database Security

↓

Infrastructure Security

↓

Monitoring

10.3 Security Architecture Model

Defense in Depth

BiteFixes uses multiple protection layers.

Security Layers

Layer 1

Identity Security

Layer 2

Application Security

Layer 3

API Security

Layer 4

Database Security

Layer 5

Infrastructure Security

Layer 6

Monitoring

10.4 Security Principles

Principle 1 — Least Privilege

Users receive only required permissions.

Example:

A technician can access assigned tickets only.

Principle 2 — Zero Trust

Every request must be validated.

Principle 3 — Data Protection

Customer information must be protected.

Principle 4 — Security by Design

Security is included during development.

10.5 Identity and Authentication Architecture

Overview

Authentication confirms user identity.

Authentication Flow

User Login



Identity Provider



Credential Validation



Access Token



Application Access

Authentication Provider

Primary:

Supabase Auth

Supported Methods

- Email/password.
- OAuth.
- Enterprise authentication.
- API authentication.

Token Architecture

Example:

```
{  
  "user_id": "123",  
  "tenant_id": "company001",  
  "role": "technician"  
}
```

Token Rules

Tokens must have:

- Expiration.
- Validation.
- Secure storage.

10.6 Authorization and Access Control

Overview

Authentication identifies the user.

Authorization defines permissions.

Role Based Access Control (RBAC)

OWNER

|

ADMIN

|

TECHNICIAN

|

SUPPORT

|

CUSTOMER

Permission Examples

Owner

Can:

- Manage company.
- Manage users.
- View reports.

Technician

Can:

- View assigned tickets.
- Update repair status.

Customer

Can:

- View own information.
- Create requests.

10.7 Multi-Tenant Security Model

Overview

Critical for SaaS architecture.

Each company must be isolated.

Tenant Isolation

Company A

Customers A

Tickets A

Data A

Company B

Customers B

Tickets B

Data B

Protection Mechanism

Primary:

`tenant_id`

combined with:

- Database policies.
- API validation.
- Access rules.

10.8 API Security Architecture

Overview

APIs are protected entry points.

API Security Controls

Includes:

- Authentication.
- Authorization.
- Input validation.
- Rate limiting.
- Logging.

Secure API Flow

Request

↓

Token Validation

↓

Permission Check

↓

Input Validation

↓

Processing



Response

API Protection

Against:

- Unauthorized access.
- Injection attacks.
- Abuse.
- Automated attacks.

10.9 Database Security Architecture

Overview

The database contains the most valuable information.

Protection Layers

Application Rules

+

Database Permissions

+

Row Level Security

+

Encryption

Supabase Security

Uses:

- Authentication integration.
- PostgreSQL permissions.
- Row Level Security.

Row Level Security Example

Concept:

```
tenant_id = current_user_tenant
```

10.10 Data Protection Architecture

Data Categories

Customer Data

Examples:

- Name.
- Contact.
- History.

Business Data

Examples:

- Services.
- Tickets.
- Pricing.

AI Data

Examples:

- Conversations.
- Intent.
- Knowledge.

Protection Requirements

Data must have:

- Ownership.
- Access rules.
- Retention policy.

10.11 Encryption Strategy

Encryption in Transit

All communication:

HTTPS / TLS

Encryption at Rest

Protect:

- Databases.
- Storage.
- Backups.

Sensitive Data

Examples:

- Credentials.
- Tokens.
- Private documents.

10.12 Secret Management

Overview

Secrets must never exist in source code.

Secret Examples

DATABASE_PASSWORD

API_KEYS

SUPABASE_SECRET

WHATSAPP_TOKEN

AI_PROVIDER_KEY

Storage

Use:

- Environment variables.
- Secret managers.

10.13 Application Security

Backend Security

Includes:

- Input validation.
- Error handling.
- Dependency updates.
- Secure coding.

Common Threats

Protection against:

- SQL injection.
- XSS.
- CSRF.
- Broken authentication.

10.14 Mobile Application Security

Flutter Security

Controls:

- Secure token storage.
- API encryption.
- Session protection.

Mobile Rules

Never store:

- Passwords.
- Private keys.
- Sensitive information.

10.15 WordPress Security

Overview

The website is a public entry point.

Protection Measures

Includes:

- Secure plugins.
- Updates.
- Limited administration access.
- API authentication.

Integration Security

WordPress communicates only through:

- Authenticated APIs.
- Validated requests.

10.16 Bity AI Security Architecture

Overview

AI requires special protection.

AI Security Risks

Examples:

- Data exposure.
- Incorrect permissions.
- Prompt manipulation.

Bitey Protection Model

User Input



Validation



Context Control



Knowledge Access Rules



AI Response

AI Data Rules

Bitey must:

- Respect tenant isolation.
- Protect private conversations.
- Avoid unauthorized knowledge access.

External AI Providers

When using:

- OpenAI.
- Gemini.

Rules:

- Send minimum required data.
- Protect customer information.
- Validate responses.

10.17 Security Monitoring

Overview

Security events must be visible.

Security Events

Monitor:

- Failed logins.
- Permission changes.
- Suspicious activity.
- API abuse.

Security Logs

Store:

- Event.

- User.
- Timestamp.
- Source.

10.18 Incident Response

Purpose

Define actions during security events.

Incident Process

Detection

↓

Containment

↓

Investigation

↓

Recovery

↓

Improvement

Incident Types

- Data exposure.
- Unauthorized access.
- Service compromise.
- AI security issue.

10.19 Security Compliance Strategy

Future compliance preparation:

- Data privacy.
- Customer protection.
- Audit readiness.

Brazil Context

BiteFixes should consider:

- LGPD principles.
- User consent.
- Data minimization.
- Data protection rights.

10.20 Volume Summary

The **Security Implementation Architecture** defines the protection framework of BiteFixes.

This volume establishes:

- Identity security.

- Authentication.
- Authorization.
- Multi-tenant protection.
- API security.
- Database security.
- Encryption.
- Secret management.
- AI security.
- Incident response.

Security becomes a fundamental layer that enables BiteFixes to operate as a trustworthy SaaS platform.

End of Volume 10

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 10

Security Implementation Architecture (SIA)

Version: 1.0

Classification: Security Architecture Documentation

Status: Draft

Table of Contents

10.1 Purpose

10.2 Security Vision

10.3 Security Architecture Model

10.4 Security Principles

10.5 Identity and Authentication Architecture

10.6 Authorization and Access Control

10.7 Multi-Tenant Security Model

10.8 API Security Architecture

10.9 Database Security Architecture

10.10 Data Protection Architecture

10.11 Encryption Strategy

10.12 Secret Management

10.13 Application Security

10.14 Mobile Application Security

10.15 WordPress Security

10.16 Bitey AI Security Architecture

10.17 Security Monitoring

10.18 Incident Response

10.19 Security Compliance Strategy

10.20 Volume Summary

10.1 Purpose

Overview

The Security Implementation Architecture defines the security framework protecting the BiteFixes ecosystem.

Security covers:

- Customer data.
- Business data.
- AI memory.
- Conversations.
- Applications.
- Infrastructure.

Security Objective

The objective is:

Protect the confidentiality, integrity, and availability of BiteFixes services while allowing secure growth as a SaaS platform.

10.2 Security Vision

Overview

Security is not an additional component.

Security is integrated into every layer.

Security Model

Users

↓

Authentication

↓

Authorization

↓

Application Security

↓

Database Security

↓

Infrastructure Security

↓

Monitoring

10.3 Security Architecture Model

Defense in Depth

BiteFixes uses multiple protection layers.

Security Layers

Layer 1

Identity Security

Layer 2

Application Security

Layer 3

API Security

Layer 4

Database Security

Layer 5

Infrastructure Security

Layer 6

Monitoring

10.4 Security Principles

Principle 1 — Least Privilege

Users receive only required permissions.

Example:

A technician can access assigned tickets only.

Principle 2 — Zero Trust

Every request must be validated.

Principle 3 — Data Protection

Customer information must be protected.

Principle 4 — Security by Design

Security is included during development.

10.5 Identity and Authentication Architecture

Overview

Authentication confirms user identity.

Authentication Flow

User Login



Identity Provider



Credential Validation



Access Token



Application Access

Authentication Provider

Primary:

Supabase Auth

Supported Methods

- Email/password.
- OAuth.
- Enterprise authentication.
- API authentication.

Token Architecture

Example:

```
{  
  "user_id": "123",  
  "tenant_id": "company001",  
  "role": "technician"  
}
```

Token Rules

Tokens must have:

- Expiration.
- Validation.
- Secure storage.

10.6 Authorization and Access Control

Overview

Authentication identifies the user.

Authorization defines permissions.

Role Based Access Control (RBAC)

OWNER

|

ADMIN

|

TECHNICIAN

|

SUPPORT

|

CUSTOMER

Permission Examples

Owner

Can:

- Manage company.
- Manage users.
- View reports.

Technician

Can:

- View assigned tickets.
- Update repair status.

Customer

Can:

- View own information.
- Create requests.

10.7 Multi-Tenant Security Model

Overview

Critical for SaaS architecture.

Each company must be isolated.

Tenant Isolation

Company A

Customers A

Tickets A

Data A

Company B

Customers B

Tickets B

Data B

Protection Mechanism

Primary:

`tenant_id`

combined with:

- Database policies.
- API validation.
- Access rules.

10.8 API Security Architecture

Overview

APIs are protected entry points.

API Security Controls

Includes:

- Authentication.
- Authorization.
- Input validation.
- Rate limiting.
- Logging.

Secure API Flow

Request

↓

Token Validation

↓

Permission Check

↓

Input Validation

↓

Processing



Response

API Protection

Against:

- Unauthorized access.
- Injection attacks.
- Abuse.
- Automated attacks.

10.9 Database Security Architecture

Overview

The database contains the most valuable information.

Protection Layers

Application Rules

+

Database Permissions

+

Row Level Security

+

Encryption

Supabase Security

Uses:

- Authentication integration.
- PostgreSQL permissions.
- Row Level Security.

Row Level Security Example

Concept:

```
tenant_id = current_user_tenant
```

10.10 Data Protection Architecture

Data Categories

Customer Data

Examples:

- Name.
- Contact.
- History.

Business Data

Examples:

- Services.
- Tickets.
- Pricing.

AI Data

Examples:

- Conversations.
- Intent.
- Knowledge.

Protection Requirements

Data must have:

- Ownership.
- Access rules.
- Retention policy.

10.11 Encryption Strategy

Encryption in Transit

All communication:

HTTPS / TLS

Encryption at Rest

Protect:

- Databases.
- Storage.
- Backups.

Sensitive Data

Examples:

- Credentials.
- Tokens.
- Private documents.

10.12 Secret Management

Overview

Secrets must never exist in source code.

Secret Examples

DATABASE_PASSWORD

API_KEYS

SUPABASE_SECRET

WHATSAPP_TOKEN

AI_PROVIDER_KEY

Storage

Use:

- Environment variables.
- Secret managers.

10.13 Application Security

Backend Security

Includes:

- Input validation.
- Error handling.
- Dependency updates.
- Secure coding.

Common Threats

Protection against:

- SQL injection.
- XSS.
- CSRF.
- Broken authentication.

10.14 Mobile Application Security

Flutter Security

Controls:

- Secure token storage.
- API encryption.
- Session protection.

Mobile Rules

Never store:

- Passwords.
- Private keys.
- Sensitive information.

10.15 WordPress Security

Overview

The website is a public entry point.

Protection Measures

Includes:

- Secure plugins.
- Updates.
- Limited administration access.
- API authentication.

Integration Security

WordPress communicates only through:

- Authenticated APIs.
- Validated requests.

10.16 Bity AI Security Architecture

Overview

AI requires special protection.

AI Security Risks

Examples:

- Data exposure.
- Incorrect permissions.
- Prompt manipulation.

Bitey Protection Model

User Input



Validation



Context Control



Knowledge Access Rules



AI Response

AI Data Rules

Bitey must:

- Respect tenant isolation.
- Protect private conversations.
- Avoid unauthorized knowledge access.

External AI Providers

When using:

- OpenAI.
- Gemini.

Rules:

- Send minimum required data.
- Protect customer information.
- Validate responses.

10.17 Security Monitoring

Overview

Security events must be visible.

Security Events

Monitor:

- Failed logins.
- Permission changes.
- Suspicious activity.
- API abuse.

Security Logs

Store:

- Event.

- User.
- Timestamp.
- Source.

10.18 Incident Response

Purpose

Define actions during security events.

Incident Process

Detection

↓

Containment

↓

Investigation

↓

Recovery

↓

Improvement

Incident Types

- Data exposure.
- Unauthorized access.
- Service compromise.
- AI security issue.

10.19 Security Compliance Strategy

Future compliance preparation:

- Data privacy.
- Customer protection.
- Audit readiness.

Brazil Context

BiteFixes should consider:

- LGPD principles.
- User consent.
- Data minimization.
- Data protection rights.

10.20 Volume Summary

The **Security Implementation Architecture** defines the protection framework of BiteFixes.

This volume establishes:

- Identity security.

- Authentication.
- Authorization.
- Multi-tenant protection.
- API security.
- Database security.
- Encryption.
- Secret management.
- AI security.
- Incident response.

Security becomes a fundamental layer that enables BiteFixes to operate as a trustworthy SaaS platform.

End of Volume 10

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 11

Testing & Quality Engineering Architecture (TQEA)

Version: 1.0

Classification: Software Quality Documentation

Status: Draft

Table of Contents

11.1 Purpose

11.2 Quality Engineering Vision

11.3 Testing Strategy Model

11.4 Testing Pyramid Architecture

11.5 Unit Testing Architecture

11.6 Backend Testing Strategy

11.7 API Testing Architecture

11.8 Database Testing Strategy

11.9 Frontend Testing Strategy

11.10 Mobile Application Testing

11.11 Integration Testing

11.12 End-to-End Testing

11.13 Bitey AI Testing Architecture

11.14 Performance Testing

11.15 Security Testing

11.16 Quality Gates

11.17 Continuous Testing Pipeline

11.18 Defect Management

11.19 Quality Metrics

11.20 Volume Summary

11.1 Purpose

Overview

The Testing & Quality Engineering Architecture defines the quality strategy for the BiteFixes ecosystem.

The purpose is to ensure:

- Reliability.
- Security.
- Performance.
- Correct functionality.
- Stable AI behavior.

Quality Objective

The objective is:

Build a predictable and trustworthy SaaS platform where every release meets technical and business quality requirements.

11.2 Quality Engineering Vision

Traditional Testing

Traditional approach:

Develop

↓

Test

↓

Release

BiteFixes Quality Model

Design

↓

Develop

↓

Automated Testing

↓

Security Validation

↓

Performance Validation



Release



Continuous Improvement

Quality Principles

Automation First

Repeated tests should be automated.

Prevention Over Correction

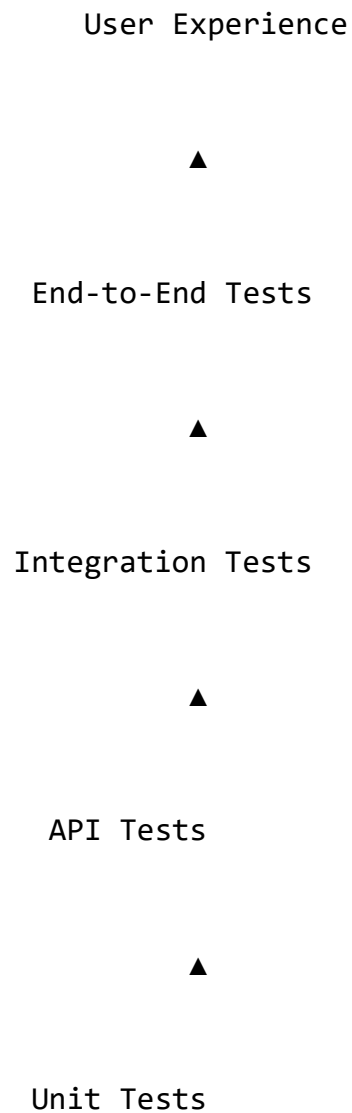
Find problems before production.

Customer Experience Focus

Technical quality must improve user experience.

11.3 Testing Strategy Model

Testing Layers



Testing Scope

BiteFixes tests:

- Backend.

- Database.
- APIs.
- Frontend.
- Mobile.
- AI Engine.
- Infrastructure.

11.4 Testing Pyramid Architecture

Overview

The testing pyramid defines test distribution.

Layer 1 — Unit Tests

Many tests.

Fast execution.

Layer 2 — Integration Tests

Validate components working together.

Layer 3 — End-to-End Tests

Validate complete user scenarios.

Example

Customer request:

Open App



Login



Send Message



Bitey Analysis



Ticket Creation



Response Received

11.5 Unit Testing Architecture

Purpose

Validate individual components.

Backend Examples

Test:

- Functions.
- Services.
- Business rules.

Example Components

`intent_service.py`

`chat_memory.py`

`ticket_service.py`

`knowledge_service.py`

Expected Validation

Example:

Input:

"Meu notebook está lento"

Expected:

`intent = upgrade_hardware`

11.6 Backend Testing Strategy

FastAPI Testing

Validate:

- Routes.
- Services.
- Models.
- Error handling.

Backend Test Areas

Authentication

Tests:

- Login.
- Permissions.
- Token validation.

Business Logic

Tests:

- Ticket creation.
- Customer management.

AI Processing

Tests:

- Intent detection.

- Knowledge retrieval.

11.7 API Testing Architecture

Purpose

Validate communication contracts.

API Test Types

Functional Tests

Example:

POST /api/v1/chat

Expected:

Correct response.

Validation Tests

Check:

- Missing data.
- Invalid formats.

Error Tests

Validate:

- 400 errors.
- 401 errors.
- 500 errors.

API Contract Example

Request:

```
{  
  "message": "Computer slow",  
  "channel": "whatsapp"  
}
```

Response:

```
{  
  "intent": "upgrade_hardware",  
  "confidence": 5  
}
```

11.8 Database Testing Strategy

Purpose

Guarantee data integrity.

Database Tests

Validate:

- Relationships.
- Constraints.

- Permissions.
- Queries.

Supabase Testing

Includes:

- Row Level Security.
- Authentication rules.
- Data isolation.

Multi-Tenant Test

Example:

Company A:

Can access:

Customer A data

Cannot access:

Customer B data

11.9 Frontend Testing Strategy

Web Testing

Validate:

- Components.
- Navigation.

- Forms.
- API communication.

Website Tests

Examples:

- Contact form.
- Chat widget.
- Customer login.

11.10 Mobile Application Testing

Flutter Testing Levels

Widget Testing

Validates:

- Buttons.
- Screens.
- Components.

Integration Testing

Validates:

- App workflows.

Device Testing

Validate:

- Android.
- iOS.
- Different screen sizes.

Mobile Scenarios

Example:

Install App

↓

Login

↓

Open Chat

↓

Receive Notification

11.11 Integration Testing

Purpose

Validate external communication.

Integrations Tested

- WhatsApp API.
- WordPress.
- Supabase.
- Payment systems.
- Notification services.

Integration Flow

External Service



BiteFixes API



Processing



Database

11.12 End-to-End Testing

Overview

Simulates real customer behavior.

Customer Scenario

Customer visits website



Starts Bitey chat



Explains problem



AI analyzes



Ticket created



Technician receives request



Customer receives update

11.13 Bitey AI Testing Architecture

Overview

AI requires specialized validation.

AI Testing Areas

Intent Accuracy

Question:

Did Bitey understand the request?

Knowledge Accuracy

Question:

Did Bitey use correct information?

Response Quality

Question:

Was the answer useful?

AI Test Dataset

Contains:

- Common questions.
- Technical problems.
- Customer conversations.

AI Evaluation Metrics

Measure:

- Accuracy.
- Confidence.
- Resolution rate.
- Escalations.

11.14 Performance Testing

Purpose

Validate system behavior under load.

Performance Tests

Load Testing

Simulate:

Many users.

Stress Testing

Find:

System limits.

Endurance Testing

Check:

Long operation periods.

Performance Targets

Examples:

API response:

< 500ms

11.15 Security Testing

Purpose

Find vulnerabilities.

Security Tests

Includes:

- Authentication testing.
- Authorization testing.
- API security.
- Dependency scanning.

Threat Examples

Test against:

- Injection.
- Unauthorized access.
- Data exposure.

11.16 Quality Gates

Overview

A release must pass defined requirements.

Production Gate

Required:

Tests Passed

+

Security Approved

+

Performance Accepted

+

Documentation Updated

Release Decision

Only approved versions reach production.

11.17 Continuous Testing Pipeline

Integration with DevSecOps:

Code Commit

↓

Automated Tests

↓

Security Scan

↓

Quality Evaluation

↓

Deployment

11.18 Defect Management

Purpose

Control problems.

Defect Lifecycle

Detected

↓

Assigned

↓

Fixed

↓

Verified

↓

Closed

Defect Priority

Critical

Service unavailable.

High

Major functionality affected.

Medium

Partial problem.

Low

Minor issue.

11.19 Quality Metrics

Important measurements:

Code Quality

- Test coverage.
- Code complexity.

Product Quality

- Failed requests.
- Customer complaints.

AI Quality

- Intent accuracy.
- Automation success.

11.20 Volume Summary

The **Testing & Quality Engineering Architecture** defines the quality control system of BiteFixes.

This volume establishes:

- Testing strategy.
- Unit testing.
- API testing.
- Database testing.
- Mobile testing.
- AI validation.
- Performance testing.
- Security testing.
- Quality gates.

Quality Engineering ensures BiteFixes evolves as a reliable, secure, and enterprise-ready SaaS platform.

End of Volume 11

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 12

Production Operations Architecture (POA)

Version: 1.0

Classification: Operations Management Documentation

Status: Draft

Table of Contents

12.1 Purpose

12.2 Production Operations Vision

12.3 Operational Architecture Model

12.4 Operations Team Structure

12.5 Service Management Model

12.6 Incident Management Architecture

12.7 Problem Management Process

12.8 Change Management Process

12.9 Release Operations

12.10 Customer Support Operations

12.11 Technical Support Workflow

12.12 SLA Architecture

12.13 Maintenance Strategy

12.14 Disaster Recovery Operations

12.15 Business Continuity Architecture

12.16 Operational Documentation

12.17 Operational Metrics

12.18 Continuous Improvement

12.19 Volume Summary

12.1 Purpose

Overview

The Production Operations Architecture defines how BiteFixes is managed after deployment.

This volume establishes the processes required to maintain:

- Availability.
- Reliability.
- Customer satisfaction.
- Operational efficiency.

Operations Objective

The objective is:

Operate BiteFixes as a professional SaaS platform with predictable service quality and controlled operational processes.

12.2 Production Operations Vision

Overview

Development creates the platform.

Operations keeps the platform reliable.

Operational Model

Development



Deployment



Production Operations



Monitoring



Improvement

Operational Principles

Reliability First

Customer services must remain available.

Controlled Changes

Every production modification must be managed.

Continuous Improvement

Operations data improves the platform.

12.3 Operational Architecture Model

Overview

Production operations connects people, processes, and technology.

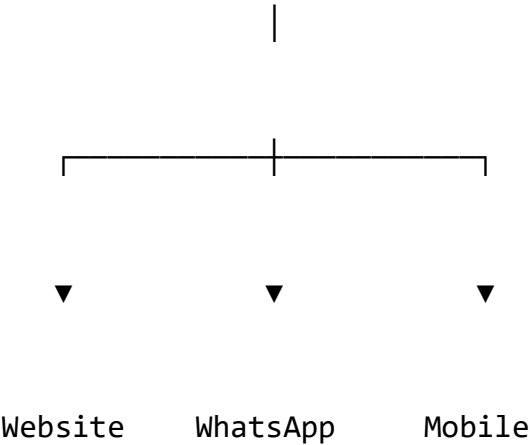
Architecture

Customers

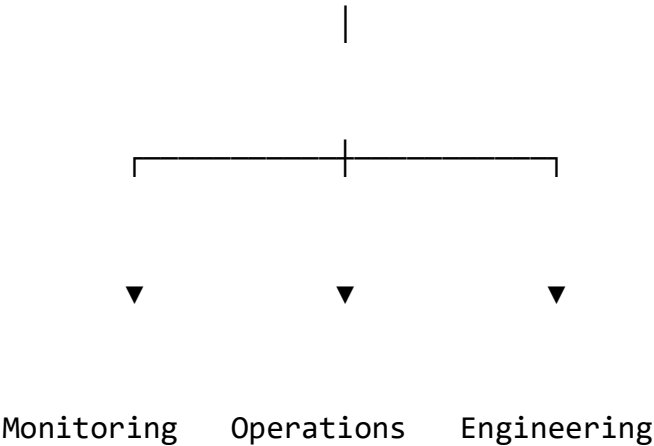
|



Support Channels



BiteFixes Platform



12.4 Operations Team Structure

Startup Stage

Initial team:

Founder / Technical Lead

|

├─ Development

├─ Support

└─ Infrastructure

Growth Stage

Future SaaS structure:

Engineering Team

Operations Team

Support Team

Security Team

AI Team

12.5 Service Management Model

Overview

BiteFixes operations follow IT Service Management principles.

Service Lifecycle

Design



Deploy



Operate



Monitor



Improve

Managed Services

Examples:

- Technical support.
- Computer repair.

- Network services.
- AI assistance.
- Customer management.

12.6 Incident Management Architecture

Purpose

Restore service quickly after failures.

Incident Flow

Detection

↓

Classification

↓

Priority Assignment

↓

Resolution

↓

Recovery

↓

Review

Incident Categories

Infrastructure Incident

Example:

Backend unavailable.

Application Incident

Example:

API failure.

Data Incident

Example:

Database problem.

AI Incident

Example:

Incorrect automated behavior.

12.7 Problem Management Process

Purpose

Prevent repeated incidents.

Problem Lifecycle

Incident

↓

Root Cause Analysis

↓

Solution

↓

Prevention

Root Cause Analysis

Methods:

- Five Whys.
- Technical analysis.
- Log investigation.

Example

Problem:

Slow API response.

Analysis:

Database query optimization required.

12.8 Change Management Process

Overview

Production changes require control.

Change Types

Standard Change

Low risk.

Example:

Documentation update.

Normal Change

Requires review.

Example:

New API feature.

Emergency Change

Critical fix.

Example:

Security vulnerability patch.

Change Flow

Request



Analysis



Approval



Implementation



Validation

12.9 Release Operations

Overview

Releases introduce new capabilities.

Release Process

Development Complete



Testing



Approval



Deployment



Monitoring

Release Documentation

Each release includes:

- Version.
- Changes.

- Known issues.
- Rollback plan.

12.10 Customer Support Operations

Overview

Customer support is integrated with Bitey AI.

Support Channels

Website Chat

WhatsApp

Mobile App

Email

Support Model

Customer



Bitey AI



Automatic Resolution



Human Support



Technical Team

12.11 Technical Support Workflow

Ticket Lifecycle

Created



Categorized



Assigned



Diagnosed



Resolved



Closed

Ticket Information

Contains:

- Customer.
- Problem.
- Priority.
- AI analysis.
- Technician notes.

12.12 SLA Architecture

Overview

Service Level Agreements define commitments.

SLA Metrics

Availability

Example:

99.9%

Response Time

Example:

Critical issue:

Immediate response.

Resolution Time

Depends on priority.

Priority Model

P1

Critical

P2

High

P3

Normal

P4

Low

12.13 Maintenance Strategy

Purpose

Maintain platform health.

Maintenance Types

Preventive Maintenance

Avoid problems.

Corrective Maintenance

Fix problems.

Evolutionary Maintenance

Improve features.

Maintenance Activities

- Updates.
- Database optimization.
- Security patches.
- Performance improvements.

12.14 Disaster Recovery Operations

Purpose

Recover after major failures.

Recovery Model

Failure



Detection



Recovery Plan



Restore Service



Validation

Recovery Components

- Database backups.
- Application backup.
- Configuration backup.

12.15 Business Continuity Architecture

Objective

Keep business operating during disruptions.

Continuity Strategy

Includes:

- Backup communication channels.
- Data recovery.
- Alternative operations.

Recovery Metrics

RTO

Recovery Time Objective.

RPO

Recovery Point Objective.

12.16 Operational Documentation

Required documents:

- Architecture manuals.
- Runbooks.
- Procedures.
- Troubleshooting guides.

Runbook Example

Problem:

Backend unavailable.

Steps:

1. Check monitoring.
2. Verify deployment.
3. Review logs.
4. Restore service.

12.17 Operational Metrics

Measure:

Technical

- Availability.
- Errors.
- Performance.

Support

- Ticket volume.
- Resolution time.
- Satisfaction.

AI

- Automation rate.
- Escalations.

12.18 Continuous Improvement

Operations creates feedback.

Improvement Cycle

Measure

↓

Analyze

↓

Improve

↓

Validate

Improvement Sources

- Customer feedback.

- Incident reports.
- Analytics.
- AI performance.

12.19 Volume Summary

The **Production Operations Architecture** defines how BiteFixes operates as a professional SaaS platform.

This volume establishes:

- Operations model.
- Incident management.
- Change management.
- Support workflows.
- SLA.
- Maintenance.
- Disaster recovery.
- Business continuity.

Operations transforms BiteFixes from a software project into a reliable technology service.

End of Volume 12

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 13

Enterprise Scaling Architecture (ESA)

Version: 1.0

Classification: Enterprise Architecture Documentation

Status: Draft

Table of Contents

13.1 Purpose

13.2 Enterprise Scaling Vision

13.3 SaaS Growth Evolution Model

13.4 Scalability Architecture Principles

13.5 Horizontal Scaling Architecture

13.6 Backend Scaling Strategy

13.7 Database Scaling Architecture

13.8 Multi-Tenant Enterprise Architecture

13.9 Microservices Evolution Strategy

13.10 Container Orchestration Architecture

13.11 Kubernetes Architecture

13.12 Multi-Region Architecture

13.13 Global Availability Model

13.14 Enterprise Integration Architecture

13.15 AI Scaling Architecture

13.16 Data Platform Evolution

13.17 Enterprise Security Scaling

13.18 Cost Optimization Strategy

13.19 Future Technology Roadmap

13.20 Volume Summary

13.1 Purpose

Overview

The Enterprise Scaling Architecture defines how BiteFixes evolves from an initial SaaS platform into an enterprise-grade technology ecosystem.

The objective is to support:

- Thousands of companies.
- Millions of conversations.
- Large amounts of technical data.
- Advanced AI automation.

Scaling Objective

The objective is:

Build an architecture capable of growing without redesigning the entire platform.

13.2 Enterprise Scaling Vision

Evolution Model

BiteFixes follows progressive growth.

Startup SaaS



Growing SaaS Platform



Enterprise SaaS



Global Technology Platform

Growth Stages

Stage 1 — MVP

Characteristics:

- Single backend.
- Managed database.
- Basic monitoring.

Stage 2 — SaaS Growth

Characteristics:

- More customers.
- Higher traffic.
- Advanced automation.

Stage 3 — Enterprise

Characteristics:

- Multi-region.
- Advanced security.
- Dedicated environments.

13.3 SaaS Growth Evolution Model

Initial Architecture

FastAPI

+

Supabase

+

Render

+

Bitey AI

Enterprise Architecture

Load Balancer

|



Multiple Backend Services

|



Distributed Data Layer

|



AI Platform Layer

13.4 Scalability Architecture Principles

Principle 1

Scale components independently.

Principle 2

Avoid single points of failure.

Principle 3

Automate infrastructure.

Principle 4

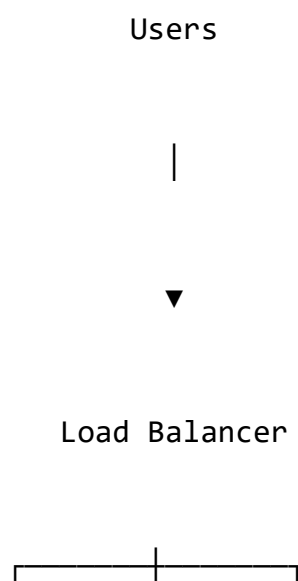
Maintain customer isolation.

13.5 Horizontal Scaling Architecture

Overview

Horizontal scaling adds more instances.

Model





API 1

API 2

API 3

Benefits

- More capacity.
- Better availability.
- Failure tolerance.

13.6 Backend Scaling Strategy

Current

Single FastAPI service.

Future

Service-based architecture.

Backend Evolution

Monolith



Modular Backend



Microservices

Possible Services

Customer Service

Handles:

- Users.
- Companies.

Ticket Service

Handles:

- Repairs.
- Requests.

AI Service

Handles:

- Bity processing.

Notification Service

Handles:

- Messages.

13.7 Database Scaling Architecture

Current

PostgreSQL Supabase.

Future Evolution

Single Database



Optimized Database



Database Replication



Distributed Data Platform

Database Scaling Techniques

Index Optimization

Improve query speed.

Read Replicas

Separate reading workload.

Partitioning

Divide large datasets.

13.8 Multi-Tenant Enterprise Architecture

Overview

Enterprise customers require stronger isolation.

Tenant Models

Shared Database

Used initially.

Database

├─ Company A

└─ Company B

└─ Company C

Dedicated Database

Enterprise option.

Company Enterprise A



Dedicated Database

Enterprise Benefits

- Isolation.
- Compliance.
- Customization.

13.9 Microservices Evolution Strategy

Purpose

Separate business capabilities.

Service Architecture

API Gateway

└ Customer Service

└ Ticket Service

└ AI Service

└ Billing Service

└ Notification Service

Migration Strategy

Do not rewrite everything.

Evolution:

Extract Services Gradually

13.10 Container Orchestration Architecture

Overview

Large systems require automated container management.

Container Model

Docker Containers



Orchestration Platform



Automatic Scaling

13.11 Kubernetes Architecture

Purpose

Manage enterprise workloads.

Kubernetes Components

Cluster

|

├─ Nodes

|

├─ Pods

|

├─ Services

|

└─ Ingress

Kubernetes Benefits

- Auto scaling.
- Self healing.
- Deployment automation.

13.12 Multi-Region Architecture

Overview

Global customers require regional infrastructure.

Model

Region A

Brazil

Region B

North America

Region C

Europe



Global Management Layer

Benefits

- Lower latency.
- Disaster protection.
- Regulatory flexibility.

13.13 Global Availability Model

Objective

Maintain service availability worldwide.

Availability Strategy

Includes:

- Multiple regions.
- Automated failover.
- Data replication.

Example

Failure:

Region A unavailable

↓

Traffic moves

↓

Region B

13.14 Enterprise Integration Architecture

Overview

Large companies require integrations.

Enterprise Integrations

Examples:

- ERP systems.
- CRM systems.
- Identity providers.
- IT management platforms.

Integration Model

Enterprise System



BiteFixes API



Platform Services

13.15 AI Scaling Architecture

Overview

Bitey becomes an AI platform.

AI Evolution

AI Assistant



AI Agent



Autonomous Service Platform

AI Scaling Components

AI Processing Layer

Handles:

- Conversations.
- Analysis.
- Recommendations.

Knowledge Layer

Handles:

- Technical knowledge.
- Customer history.

Learning Layer

Improves:

- Accuracy.
- Automation.

13.16 Data Platform Evolution

Overview

Data becomes a strategic asset.

Future Data Architecture

Operational Database



Data Warehouse



Analytics Platform



AI Intelligence

Data Capabilities

- Business intelligence.
- Predictive maintenance.
- Customer insights.

13.17 Enterprise Security Scaling

Enterprise security requires:

- Advanced identity management.
- Audit logs.
- Compliance controls.
- Security monitoring.

Enterprise Features

Possible:

- SSO.
- MFA.
- Advanced permissions.
- Dedicated environments.

13.18 Cost Optimization Strategy

Objective

Scale efficiently.

Cost Controls

- Resource optimization.
- Automatic scaling.
- Usage monitoring.

Cloud Economics

Balance:

Performance

+

Reliability

+

Cost

13.19 Future Technology Roadmap

Possible future additions:

- Kubernetes.
- AI agents.
- Edge computing.
- Advanced analytics.
- Autonomous diagnostics.

13.20 Volume Summary

The **Enterprise Scaling Architecture** defines the long-term evolution of BiteFixes.

This volume establishes:

- SaaS growth stages.
- Horizontal scaling.
- Backend evolution.
- Database scaling.
- Multi-tenancy.
- Microservices.
- Kubernetes.
- Multi-region architecture.
- Enterprise AI evolution.

This architecture allows BiteFixes to evolve from a local technology service platform into a global intelligent SaaS ecosystem.

End of Volume 13

BiteFixes Technical Implementation Blueprint

BiteFixes Technical Implementation Blueprint

Volume 14

AI Advanced Evolution Architecture (AIEA)

Version: 1.0

Classification: Artificial Intelligence Architecture Documentation

Status: Draft

Table of Contents

14.1 Purpose

14.2 Bitey AI Evolution Vision

14.3 AI Platform Evolution Model

14.4 Advanced AI Architecture

14.5 AI Agent Architecture

14.6 Multi-Agent System Design

14.7 AI Memory Evolution

14.8 Knowledge Intelligence Architecture

14.9 Predictive Diagnostics Architecture

14.10 Autonomous Workflow Architecture

14.11 Human + AI Collaboration Model

14.12 AI Decision Engine

14.13 AI Security Architecture

14.14 AI Evaluation Framework

14.15 AI Improvement Lifecycle

14.16 Enterprise AI Capabilities

14.17 Future AI Roadmap

14.18 Volume Summary

14.1 Purpose

Overview

The AI Advanced Evolution Architecture defines the future development path of Bitey AI.

The objective is to evolve Bitey from:

AI Assistant



AI Support Engine



AI Agent Platform



Autonomous Technical Intelligence

AI Objective

The objective is:

Create an intelligent technical ecosystem capable of understanding problems, assisting users, automating workflows, and continuously improving.

14.2 Bitey AI Evolution Vision

Current State

Bitey provides:

- Intent detection.
- Knowledge retrieval.
- Ticket creation.
- Conversation memory.

Future State

Bitey becomes:

- Technical assistant.
- Diagnostic agent.
- Business assistant.
- Automation coordinator.

Evolution Model

Phase 1

Conversational AI



Phase 2

Technical AI Assistant



Phase 3

AI Agents



Phase 4

Autonomous AI Platform

14.3 AI Platform Evolution Model

Current Architecture

User



Bitey Engine



Knowledge Base



Response

Advanced Architecture

User



AI Orchestrator



Specialized Agents



Tools + Knowledge + Memory



Action Execution

14.4 Advanced AI Architecture

Overview

Future Bitey architecture contains multiple intelligence layers.

AI Layers

Interaction Layer



Reasoning Layer



Knowledge Layer



Memory Layer



Action Layer

Interaction Layer

Responsible for:

- Chat.
- Voice.
- WhatsApp.
- Mobile.

Reasoning Layer

Responsible for:

- Understanding.
- Planning.
- Decision support.

Knowledge Layer

Contains:

- Technical documentation.
- Solutions.
- Procedures.

Memory Layer

Contains:

- Customer history.
- Previous problems.
- Preferences.

Action Layer

Executes:

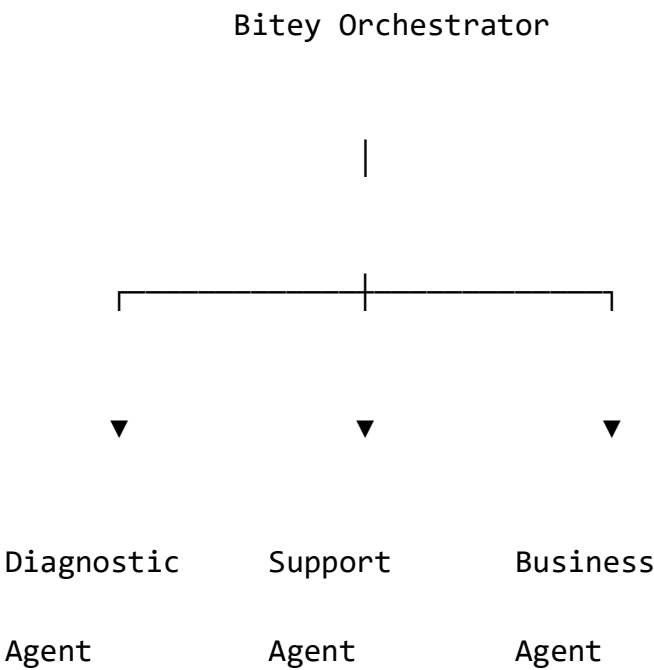
- Ticket creation.
- Notifications.
- Workflows.

14.5 AI Agent Architecture

Overview

Bitey evolves into specialized AI agents.

Agent Model



Diagnostic Agent

Purpose:

Analyze technical problems.

Examples:

- Computer failures.
- Network issues.
- Hardware diagnostics.

Support Agent

Purpose:

Customer communication.

Responsibilities:

- Answer questions.
- Guide users.
- Create tickets.

Business Agent

Purpose:

Help operations.

Responsibilities:

- Reports.
- Recommendations.
- Customer insights.

14.6 Multi-Agent System Design

Overview

Multiple agents collaborate.

Agent Communication

User Request



Bitey Orchestrator



Select Agent



Execute Task



Return Result

Example

Customer:

"My computer is slow"

Process:

Diagnostic Agent



Hardware Analysis

Support Agent



Customer Explanation

Ticket Agent



Creates Service Request

14.7 AI Memory Evolution

Overview

Memory is one of Bitey's most important capabilities.

Current Memory

Stores:

- Conversations.
- Customer history.
- Tickets.

Future Memory Architecture

Short Term Memory

+

Long Term Memory

+

Business Memory

+

Technical Memory

Short Term Memory

Current conversation context.

Long Term Memory

Historical interactions.

Business Memory

Customer preferences and relationships.

Technical Memory

Previous diagnostics and solutions.

14.8 Knowledge Intelligence Architecture

Overview

Knowledge evolves from static information into intelligent knowledge.

Current

Database:

Question

Answer

Category

Tags

Future

Intelligent Knowledge Graph:

Problem



Cause



Solution



Service

Knowledge Capabilities

Bitey can:

- Find similar problems.
- Recommend solutions.
- Improve documentation.

14.9 Predictive Diagnostics Architecture

Overview

Future Bitey predicts failures.

Predictive Model

Data Collection



Pattern Analysis



Prediction



Recommendation

Examples

Prediction:

"SSD health decreasing"

Recommendation:

"Schedule replacement."

14.10 Autonomous Workflow Architecture

Overview

Bitey automates repetitive processes.

Workflow Example

Customer Request



AI Analysis



Create Ticket



Assign Technician



Notify Customer



Follow Resolution

Automation Areas

- Scheduling.
- Notifications.
- Reports.
- Customer follow-up.

14.11 Human + AI Collaboration Model

Overview

AI assists humans.

AI does not replace critical decisions.

Collaboration Model

AI Analysis



Human Validation



Final Decision

Technician Assistance

Bitey provides:

- Diagnostic suggestions.
- Repair history.
- Recommended actions.

14.12 AI Decision Engine

Purpose

Select the correct action.

Decision Process

Input



Context Analysis



Rules



AI Reasoning



Action Recommendation

Decision Factors

- Customer history.
- Technical data.
- Business rules.
- Confidence level.

14.13 AI Security Architecture

AI Protection Requirements

Protect:

- Conversations.
- Customer data.
- Internal knowledge.

AI Security Controls

Includes:

- Permission-aware responses.
- Data filtering.
- Audit logs.
- Prompt protection.

14.14 AI Evaluation Framework

Purpose

Measure AI quality.

Metrics

Accuracy

Did Bitey understand?

Helpfulness

Did the answer solve the problem?

Automation Rate

How many cases solved automatically?

Customer Satisfaction

User experience measurement.

14.15 AI Improvement Lifecycle

Continuous improvement:

Collect Data

↓

Analyze Results

↓

Improve Models



Validate



Deploy Improvements

14.16 Enterprise AI Capabilities

Future enterprise features:

- Private AI environments.
- Company-specific knowledge.
- Custom agents.
- Advanced analytics.
- AI governance.

14.17 Future AI Roadmap

Evolution:

AI Chat



AI Support Engine



AI Technical Assistant



AI Agents



Autonomous Service Platform

14.18 Volume Summary

The **AI Advanced Evolution Architecture** defines the future intelligence layer of BiteFixes.

This volume establishes:

- Bitey AI evolution.
- Agent architecture.
- Multi-agent systems.
- Advanced memory.
- Knowledge intelligence.
- Predictive diagnostics.
- Autonomous workflows.
- Human-AI collaboration.

Bitey becomes the intelligence core that differentiates BiteFixes from traditional technical support platforms.

End of Volume 14

**BiteFixes Technical Implementation
Blueprint**

**BiteFixes Technical Implementation
Blueprint**

Volume 15

**Complete Implementation Roadmap
(CIR)**

Version: 1.0

Classification: Strategic Implementation Documentation

Status: Draft

Table of Contents

15.1 Purpose

15.2 Implementation Vision

15.3 Current Platform Status

15.4 Development Strategy

15.5 Phase 1 — Foundation MVP

15.6 Phase 2 — Intelligent Support Platform

15.7 Phase 3 — SaaS Business Platform

15.8 Phase 4 — Enterprise Platform

15.9 Phase 5 — AI Autonomous Ecosystem

15.10 Technical Priority Matrix

15.11 Development Sequence

15.12 Business Evolution Model

15.13 Team Evolution Strategy

15.14 Long-Term Technology Roadmap

15.15 Final Architecture Vision

15.16 Volume Summary

15.1 Purpose

Overview

The Complete Implementation Roadmap defines the execution strategy for transforming BiteFixes from its current development state into a scalable intelligent SaaS platform.

Objective

The objective is:

Provide a realistic path from MVP implementation to enterprise-scale technology company.

15.2 Implementation Vision

BiteFixes development follows progressive evolution.

Evolution Model

Technical Service Platform



Digital Support Platform



AI SaaS Platform



Enterprise Intelligence Platform



Global Technology Ecosystem

15.3 Current Platform Status

Current Achievements

BiteFixes already contains the foundation:

Backend

Implemented:

- FastAPI architecture.
- Service-based structure.
- API endpoints.
- Customer management.
- Ticket system.

Database

Implemented:

- Supabase integration.
- PostgreSQL model.

- Knowledge database.
- Conversation storage.

Bitey AI

Implemented:

- Intent detection.
- Knowledge retrieval.
- Memory.
- Automatic ticket generation.

Integrations

Architecture defined:

- WordPress.
- WhatsApp.
- Flutter application.

15.4 Development Strategy

The implementation follows:

Complete Core



Validate Market



Improve Product



Scale Infrastructure



Enterprise Evolution

15.5 Phase 1 — Foundation MVP

Objective

Create a functional service platform.

Components

Backend

Complete:

- FastAPI.
- Authentication.
- Customers.
- Tickets.
- Services.

Database

Complete:

- Production schema.

- Security rules.
- Backup.

Bitey AI

Complete:

- Knowledge search.
- Intent detection.
- Basic conversations.

Website

Implement:

- Service pages.
- Contact.
- AI chat widget.

MVP Result

Customer can:

Contact BiteFixes



Talk with Bitey



Create Request



Track Service

15.6 Phase 2 — Intelligent Support Platform

Objective

Transform support into an AI-assisted experience.

New Capabilities

Advanced Bitey

Add:

- Better memory.
- Better intent detection.
- Multilingual responses.

Customer Portal

Add:

- Login.
- Tickets.
- History.
- Notifications.

Technician Portal

Add:

- Assigned work.
- Status updates.
- Technical notes.

Result

BiteFixes becomes:

An intelligent technical support platform.

15.7 Phase 3 — SaaS Business Platform

Objective

Convert platform into a commercial SaaS product.

SaaS Features

Multi-Tenant Architecture

Support:

- Multiple companies.
- Isolated data.
- Company users.

Subscription System

Add:

- Plans.
- Billing.
- Usage limits.

Analytics

Add:

- Business dashboards.
- Service reports.
- AI metrics.

Result

BiteFixes becomes:

A SaaS product for technical service companies.

15.8 Phase 4 — Enterprise Platform

Objective

Support large organizations.

Enterprise Features

Advanced Security

Includes:

- MFA.
- SSO.
- Audit logs.

Infrastructure

Includes:

- Kubernetes.
- Multi-region.
- Advanced monitoring.

Integrations

Includes:

- ERP.
- CRM.
- IT management systems.

Result

BiteFixes becomes:

Enterprise-ready intelligent support infrastructure.

15.9 Phase 5 — AI Autonomous Ecosystem

Objective

Transform Bitey into an autonomous technical intelligence platform.

Capabilities

AI Agents

Examples:

- Diagnostic Agent.
- Support Agent.
- Business Agent.

Predictive Intelligence

Examples:

- Failure prediction.
- Maintenance recommendations.

Autonomous Operations

Examples:

- Automatic workflows.
- Smart scheduling.
- Continuous optimization.

Result

BiteFixes becomes:

An AI-powered technology operations ecosystem.

15.10 Technical Priority Matrix

Priority 1 — Critical

Build first:

- Backend stability.
- Database.
- Bitey Core.
- Authentication.
- Ticket system.

Priority 2 — Important

Build next:

- Customer portal.
- WhatsApp integration.
- Mobile application.
- Notifications.

Priority 3 — Growth

Build later:

- SaaS billing.

- Analytics.
- Enterprise features.

Priority 4 — Future

Long term:

- AI agents.
- Kubernetes.
- Global infrastructure.

15.11 Development Sequence

Recommended order:

1

Backend Stabilization



2

Production Database



3

Authentication



4

Bitey Improvement



5

Channels Integration



6

Customer Portal



7

Mobile App



8

SaaS Features



15.12 Business Evolution Model

Stage 1

Local technical services.

Stage 2

Digital service platform.

Stage 3

SaaS provider.

Stage 4

AI technology company.

Business Transformation

Repair Company



Technology Platform



Software Company



AI Enterprise

15.13 Team Evolution Strategy

Initial Team

Founder:

Responsibilities:

- Development.
- Infrastructure.
- Support.

Growth Team

Future:

Engineering

Develop platform.

Support

Manage customers.

Sales

Acquire companies.

AI Team

Improve intelligence.

15.14 Long-Term Technology Roadmap

Future technologies:

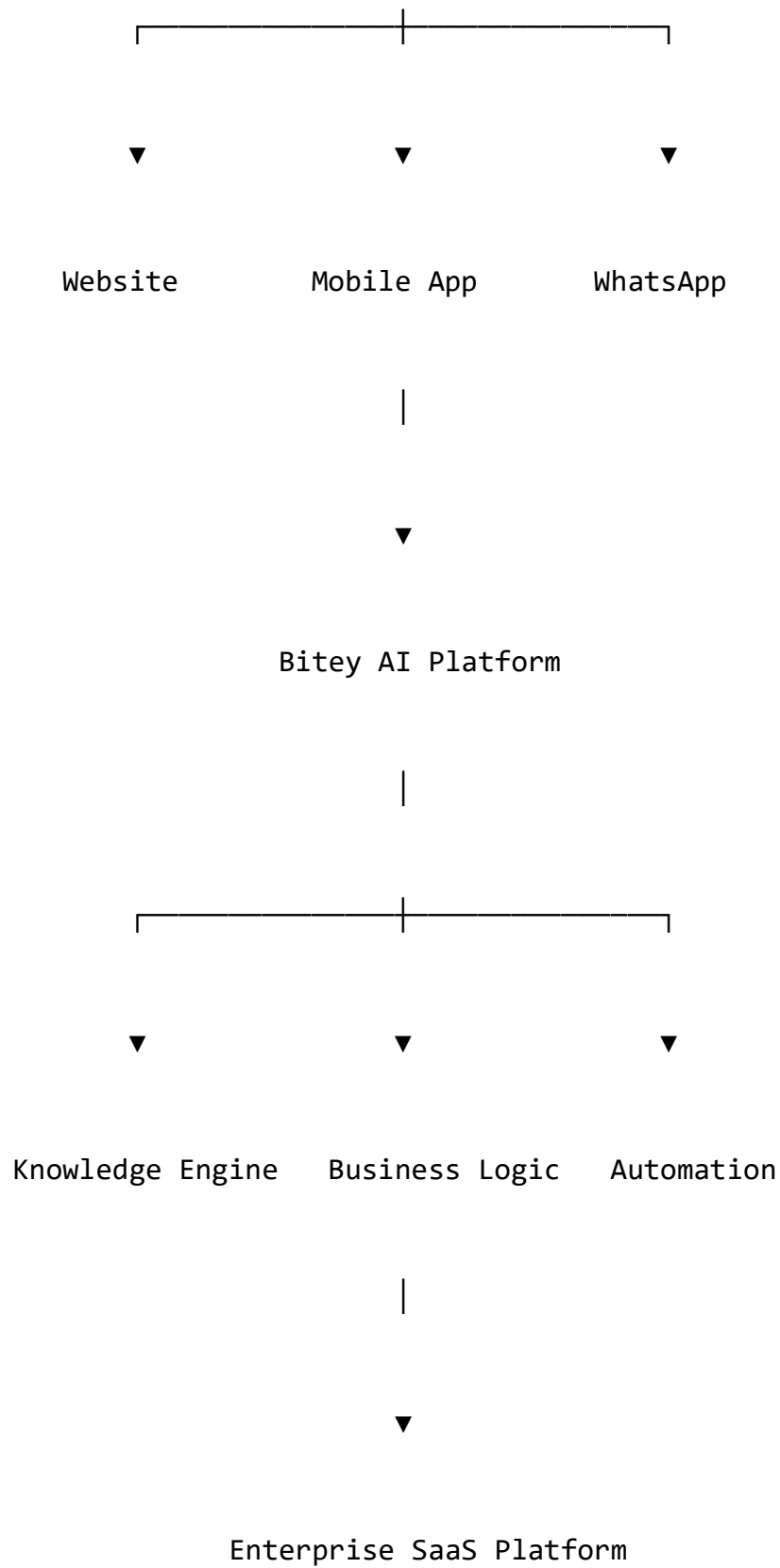
- Advanced AI agents.
- Machine learning models.
- Predictive maintenance.
- Cloud-native architecture.
- Global SaaS infrastructure.

15.15 Final Architecture Vision

The complete BiteFixes ecosystem:

Customers

|



15.16 Volume Summary

The **Complete Implementation Roadmap** defines the execution path for BiteFixes.

The complete blueprint establishes:

- Platform foundation.
- Backend architecture.
- Database architecture.
- Bitey AI engine.
- Integrations.
- Frontend.
- Cloud infrastructure.
- DevSecOps.
- Monitoring.
- Security.
- Testing.
- Operations.
- Enterprise scaling.
- AI evolution.
- Implementation roadmap.

Final Statement

BiteFixes is designed not only as a repair service platform, but as an intelligent SaaS ecosystem where:

- Technical services,
- Artificial intelligence,
- Automation,
- Customer experience,
- Business intelligence,

converge into a scalable technology platform.

Bitey AI is the central intelligence layer that connects every customer, technician, service process, and business decision.

End of Volume 15

End of BiteFixes Technical Implementation Blueprint

Complete Architecture Series — Volumes 01 to 15